Software Engineering Project

Software Requirements Specification Document

Project Title : CP Performance Analyzer

Group Number :

Group Members Details :

Member Name	Registration Number
Raghav Setia	20243227
Rahul Kumar	20243228
Rajat Kumar Gupta	20243229
Shreyansh Pal	20243588

Course Name : Software Engineering and Project Management

Section : E

Instructor Name : Dr. Satarupa Chakrabarti

The Problem :

Competitive programming platforms such as ***Codeforces*** provide users with information about their **submissions, contest ratings,** and **solved problems.** However, this information is often scattered across different sections and does not directly provide a comprehensive analysis of a programmer's performance.

As a result, students and competitive programmers face several challenges:

- **Lack of Personalized Insights:** Difficulty in identifying individual strengths and weaknesses across different problem-solving topics.
- **Unclear Progress Tracking:** Difficulty in understanding performance trends and improvements over time.
- **Inefficient Practice:** Students often solve problems randomly without knowing which topics or difficulty levels require more attention.
- **Fragmented Information:** Contest performance, submission history, and problem-solving statistics are not presented together in an easily understandable analytical dashboard.

Our project, CP Performance Analyzer, aims to address these challenges by developing a web-based platform that analyzes competitive programming data and presents meaningful performance insights through a centralized dashboard.

Stakeholder Identification

Stakeholders	Role
Competitive Programmers	Primary Users who analyze their performance
Beginners/Students	Identify topics requiring practice
College Coding Clubs	Interpret members performance and progress
Development Team	Develop, Test, Deploy and Maintain
Codeforces Platform	External Data Provider through its API

Requirements Specification

The requirements specification defines the functional capabilities, quality attributes, and development priorities of the CP Performance Analyzer. These requirements guide the design, implementation, and testing of the system.

1. Functional Requirements

Functional requirements describe **what the system must do**.

ID	Requirement	Description	Priority
FR-01	User Authentication	The system shall allow users to register, log in, and access their accounts securely.	Must Have
FR-02	Profile Integration	The system shall allow users to link their Codeforces handle and retrieve the corresponding profile data.	Must Have
FR-03	Data Retrieval	The system shall fetch contest history, submissions, problem details, and rating information from the Codeforces API.	Must Have
FR-04	Performance Dashboard	The system shall display important programming statistics in a centralized dashboard.	Must Have
FR-05	Topic-wise Analysis	The system shall analyze problem-solving performance across different programming topics.	Must Have

FR-06	Difficulty-wise Analysis	The system shall categorize solved problems and submission performance according to problem difficulty.	Must Have
FR-07	Rating Visualization	The system shall display contest rating progression using charts and graphs.	Must Have
FR-08	Submission Analysis	The system shall calculate relevant statistics, such as accepted submissions and acceptance rate.	Must Have
FR-09	Progress Insights	The system shall identify performance trends and potential weak areas using predefined analytical rules.	Should Have
FR-10	Report History	The system shall allow users to save and retrieve previously generated performance reports.	Should Have
FR-11	Error Handling	The system shall display appropriate messages for invalid handles, unavailable API data, and failed requests.	Must Have

2. Non-Functional Requirements

Non-functional requirements describe **how well the system must operate**.

ID	Requirement	Description
NFR-01	Performance	The system should load the dashboard and display available analytical results within a reasonable response time under normal usage.
NFR-02	Reliability	The system should handle API failures, missing data, and invalid inputs without crashing.
NFR-03	Security	Passwords must be securely handled, protected routes must require authentication, and sensitive session information must not be exposed.
NFR-04	Usability	The interface should present statistics and visualizations clearly, allowing users to understand their performance with minimal effort.
NFR-05	Correctness	Analytical results must be derived using consistent, predefined rules and validated calculations.
NFR-06	Maintainability	The application should use modular components so that data retrieval, analysis, storage, and presentation can be modified independently.
NFR-07	Scalability	The architecture should support additional users and future analytical features without requiring a complete redesign.

NFR-08	Compatibility	The web application should work on commonly used modern browsers and adapt to different screen sizes.
NFR-09	Data Integrity	Stored profiles and reports should remain associated with the correct user and should not be unintentionally corrupted.
NFR-10	Availability	The deployed application should remain accessible whenever its hosting infrastructure and external dependencies are operational.

3. Prioritisation of Requirements (MoSCoW Prioritization)

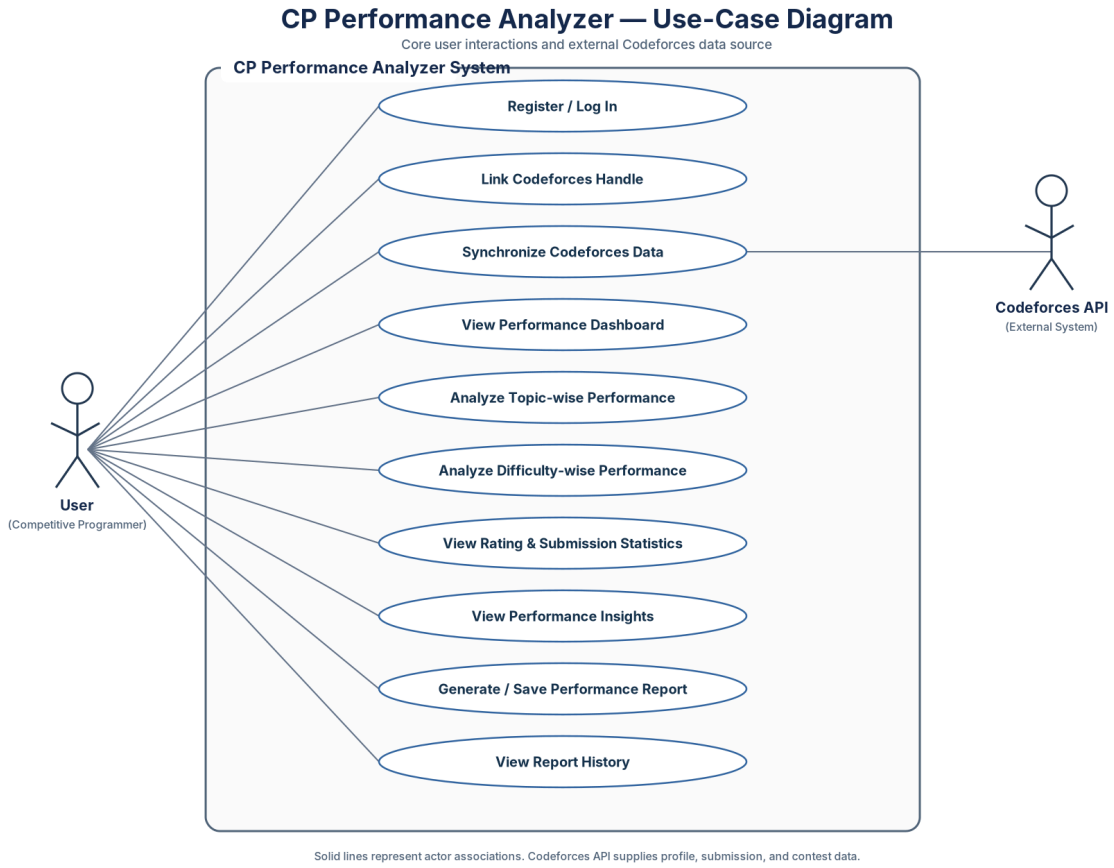
Backlog item	Priority	Iteration
Handle validation and Codeforces integration	Must	1
Submission normalization	Must	1
Topic weakness analysis	Must	1
Difficulty and verdict analytics	Must	1
Interactive dashboard	Must	1

Unit and integration testing	Must	1
Rating trends and saved reports	Should	2
Problem recommendations	Could	2
AI coaching	Won't for current evaluation	3
Multi-platform analytics	Won't for current evaluation	3

Prioritisation Rationale

Expected outcome: A working system that retrieves Codeforces data, analyses programming performance using deterministic rules, and presents the results through an understandable dashboard.

Since the project follows the **Evolutionary Development Model**, priorities can be reviewed at the end of each iteration based on testing results, user feedback, available time, and implementation feasibility.



User Stories:

US-01: Analyze a Codeforces handle

User story: As a competitive programmer, I want to enter my Codeforces handle so that I can analyze my programming performance.

Acceptance criteria:

- The system accepts a non-empty, validly formatted handle.
- The system requests the relevant public Codeforces data.
- The system shows the analysis when the data is retrieved successfully.

- Invalid handles and API failures produce clear error messages instead of a fake success result.

US-02: View a performance summary

User story: As a competitive programmer, I want to see a summary of my attempted and solved problems so that I can understand my overall practice activity.

Acceptance criteria:

- The summary displays attempted and solved problem counts.
- Multiple submissions for the same problem count as one attempted problem.
- A problem is counted as solved if it has at least one accepted (OK) submission.
- An empty submission history produces a valid empty state.

US-03: Identify topic-wise strengths and weaknesses

User story: As a competitive programmer, I want to view topic-wise statistics so that I can identify topics where my success rate is comparatively low.

Acceptance criteria:

- The system displays attempted problems, solved problems, and solve rate by topic.
- A problem associated with multiple tags is counted once under each relevant tag.
- Topics with very few attempts are not presented as definitive weaknesses.

US-04: Analyze performance by problem difficulty

User story: As a competitive programmer, I want to compare my performance across problem-rating ranges so that I can understand which difficulty levels I find challenging.

Acceptance criteria:

- Problems are grouped using their problem rating, not the user's current contest rating.
- Each rating range displays meaningful attempt and solve statistics.
- Problems without a rating are excluded from rating buckets but remain included in overall totals.

US-05: Review submission verdicts

User story: As a competitive programmer, I want to view my submission verdict distribution so that I can understand recurring errors.

Acceptance criteria:

- The system displays counts for accepted submissions, wrong answers, time-limit errors, runtime errors, and other supported verdicts.
- Verdict counts represent submissions, not unique problems.
- Unknown verdicts are handled without crashing the analysis.

US-06: Find problems for upsolving

User story: As a competitive programmer, I want to view problems I attempted but have not solved so that I can revisit them later.

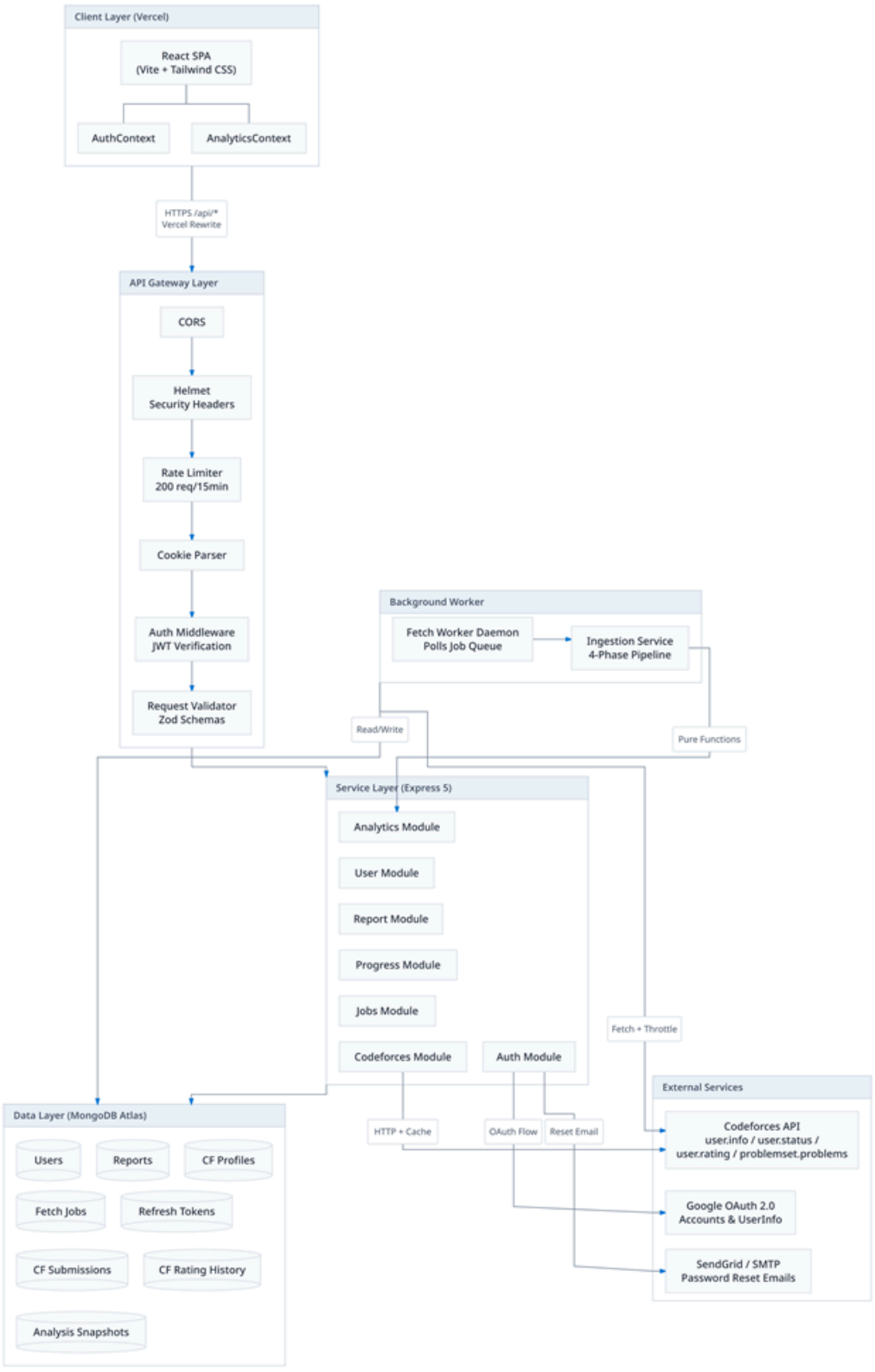
Acceptance criteria:

- The system identifies problems with submissions but no accepted verdict.
- Each problem appears only once in the upsolving list.
- A valid Codeforces problem link is displayed whenever sufficient problem identifiers are available.
- The list is empty when all attempted problems have been solved.

4. Architecture diagram

CP Performance Analyzer follows a **Modular Monolith** architecture with clear layer separation. The frontend is a React Single-Page Application deployed independently on Vercel, communicating with an Express REST API backend hosted on Render. A background worker daemon handles long-running Codeforces data ingestion asynchronously via a MongoDB-backed job queue.

High-Level System Architecture


Client Layer (Vercel)
React SPA
(Vite + Tailwind CSS)
AuthContext
AnalyticsContext
HTTPS /api/*
Vercel Rewrite
API Gateway Layer
CORS
Helmet
Security Headers
Rate Limiter
200 req/15min
Cookie Parser
Auth Middleware
JWT Verification
Request Validator
Zod Schemas
Background Worker
Fetch Worker Daemon
Polls Job Queue
Ingestion Service
4-Phase Pipeline
Read/Write
Pure Functions
Service Layer (Express 5)
Analytics Module
User Module
Report Module
Progress Module
Jobs Module
Codeforces Module
Auth Module
Fetch + Throttle
External Services
Codeforces API
user.info / user.status /
user.rating / problemset.problems
Google OAuth 2.0
Accounts & UserInfo
SendGrid / SMTP
Password Reset Emails
HTTP + Cache
OAuth Flow
Reset Email
Data Layer (MongoDB Atlas)
Users
Reports
CF Profiles
Fetch Jobs
Refresh Tokens
CF Submissions
CF Rating History
Analysis Snapshots

Layered Responsibility Model

Layer	Responsibility	Key Principle
Client	Rendering, navigation, local state, session caching, toast notifications	Code-split lazy loading; 3-tier data hydration for zero-flash UX
API Gateway	Security, rate limiting, cookie/token parsing, schema validation	Fail fast — reject malformed requests before they reach business logic
Service	Business workflows, orchestration, data transformation	Thin controllers; business rules live in service files
Analytics Engine	Deterministic scoring, aggregation, ranking	Pure functions with no side effects; every score is explainable
Worker	Async data ingestion, retry, throttling	MongoDB-backed job queue with lease locking and stale recovery
Data	Persistence, indexing, TTL expiration	Compound indexes for query performance; TTL indexes for auto-cleanup
External	Codeforces data, OAuth identity, email delivery	In-memory coalescing and stale-cache fallback for resilience

---

5. Database/data model

Entity-Relationship Diagram



Entity Attributes

User

Field	Type	Key	Notes
_id	ObjectId	PK	—
name	String	—	required
email	String	UK	unique, lowercase
passwordHash	String	—	select:false
googleId	String	—	select:false, sparse unique
emailVerified	Boolean	—	default:false
profileImageUrl	String	—	—
passwordResetTokenHash	String	—	select:false
passwordResetExpiresAt	Date	—	select:false
codeforcesHandle	String	—	—
targetRating	Number	—	800-4000, default:1600
preferredPracticeMinutes	Number	—	enum:[30,60,90,120]

difficultTopics	Array	—	String[]
preferences	Object	—	nested sub-document

Report

Field	Type	Key	Notes
_id	ObjectId	PK	—
userId	ObjectId	FK	ref:User
handle	String	—	—
reportType	String	—	enum:[manual,weekly]
profile	Object	—	snapshot
summary	Object	—	metrics snapshot
topicAnalysis	Array	—	weakness scores
ratingAnalysis	Array	—	band breakdowns
verdictAnalysis	Object	—	verdict distribution
upsolvingProblems	Array	—	prioritized list

weeklyComparison	Object	—	week-over-week deltas
source	String	—	—
generatedAt	Date	—	—
isSaved	Boolean	—	default:true

RefreshToken

Field	Type	Key	Notes
_id	ObjectId	PK	—
userId	ObjectId	FK	ref:User
tokenHash	String	—	SHA-256 hashed
familyId	String	—	UUID for rotation tracking
expiresAt	Date	—	TTL index, auto-delete
revokedAt	Date	—	null if active
replacedByHash	String	—	rotation chain
userAgent	String	—	—
ip	String	—	—

CFProfile

Field	Type	Key	Notes
_id	ObjectId	PK	—
handle	String	UK	unique
firstName	String	—	—

lastName	String	—	—
country	String	—	—
organization	String	—	—
rating	Number	—	—
maxRating	Number	—	—
rank	String	—	—
maxRank	String	—	—
friendOfCount	Number	—	—
avatar	String	—	—
registeredAt	Date	—	—
lastFetchedAt	Date	—	—

CFSubmission

Field	Type	Key	Notes
_id	ObjectId	PK	—
handle	String	—	indexed
submissionId	Number	—	unique per handle
problemKey	String	—	contestId-index
contestId	Number	—	—
index	String	—	—
name	String	—	—
rating	Number	—	—
tags	Array	—	lowercase, trimmed
verdict	String	—	—
language	String	—	—
createdAt	Date	—	—

passedTestCount	Number	—	—
participantType	String	—	—
url	String	—	—

CFRatingHistory

Field	Type	Key	Notes
_id	ObjectId	PK	—
handle	String	—	indexed
contestId	Number	—	unique per handle
contestName	String	—	—
rank	Number	—	—
oldRating	Number	—	—
newRating	Number	—	—
changedAt	Date	—	—

FetchJob

Field	Type	Key	Notes
_id	ObjectId	PK	—
handle	String	—	—
userId	ObjectId	FK	ref:User
type	String	—	enum:[init,sync]

status	String	—	enum:[PENDING,RUNNING,COMPLETED,FAILED]
attempts	Number	—	default:0
maxAttempts	Number	—	default:5
priority	Number	—	default:0
claimedBy	String	—	worker identity
claimedAt	Date	—	—
expiresAt	Date	—	—
progress	Object	—	phase,done,total
error	Object	—	message,phase
result	Object	—	snapshotId,submissionCount

AnalysisSnapshot

Field	Type	Key	Notes
_id	ObjectId	PK	—
handle	String	—	lowercase, indexed
userId	ObjectId	—	—
submissionCount	Number	—	—
latestSubmissionId	Number	—	—
analysis	Object	—	Mixed - full analytics result
generatedAt	Date	—	—
expiresAt	Date	—	TTL index, 24h auto-delete

Index Strategy

Collection	Index	Type	Purpose
User	{ email: 1 }	Unique	Fast login lookups, duplicate prevention
User	{ googleId: 1 }	Unique, Sparse	OAuth identity linking
Report	{ userId: 1, generatedAt: -1 }	Compound	User's reports in reverse chronological order
Report	{ handle: 1, generatedAt: -1 }	Compound	Handle-based report lookups
RefreshToken	{ expiresAt: 1 }	TTL	Automatic expired token cleanup
CFSubmission	{ handle: 1, submissionId: 1 }	Unique	Idempotent bulk upserts
CFSubmission	{ handle: 1, createdAt: -1 }	Compound	Chronological submission queries
CFRatingHistory	{ handle: 1, contestId: 1 }	Unique	Idempotent rating change upserts
CFProfile	{ handle: 1 }	Unique	One profile per CF user
FetchJob	{ status: 1, priority: -1, createdAt: 1 }	Compound	Efficient queue polling
AnalysisSnapshot	{ handle: 1, generatedAt: -1 }	Compound	Latest snapshot retrieval
AnalysisSnapshot	{ expiresAt: 1 }	TTL	24-hour auto-expiration

User Preferences Sub-Document

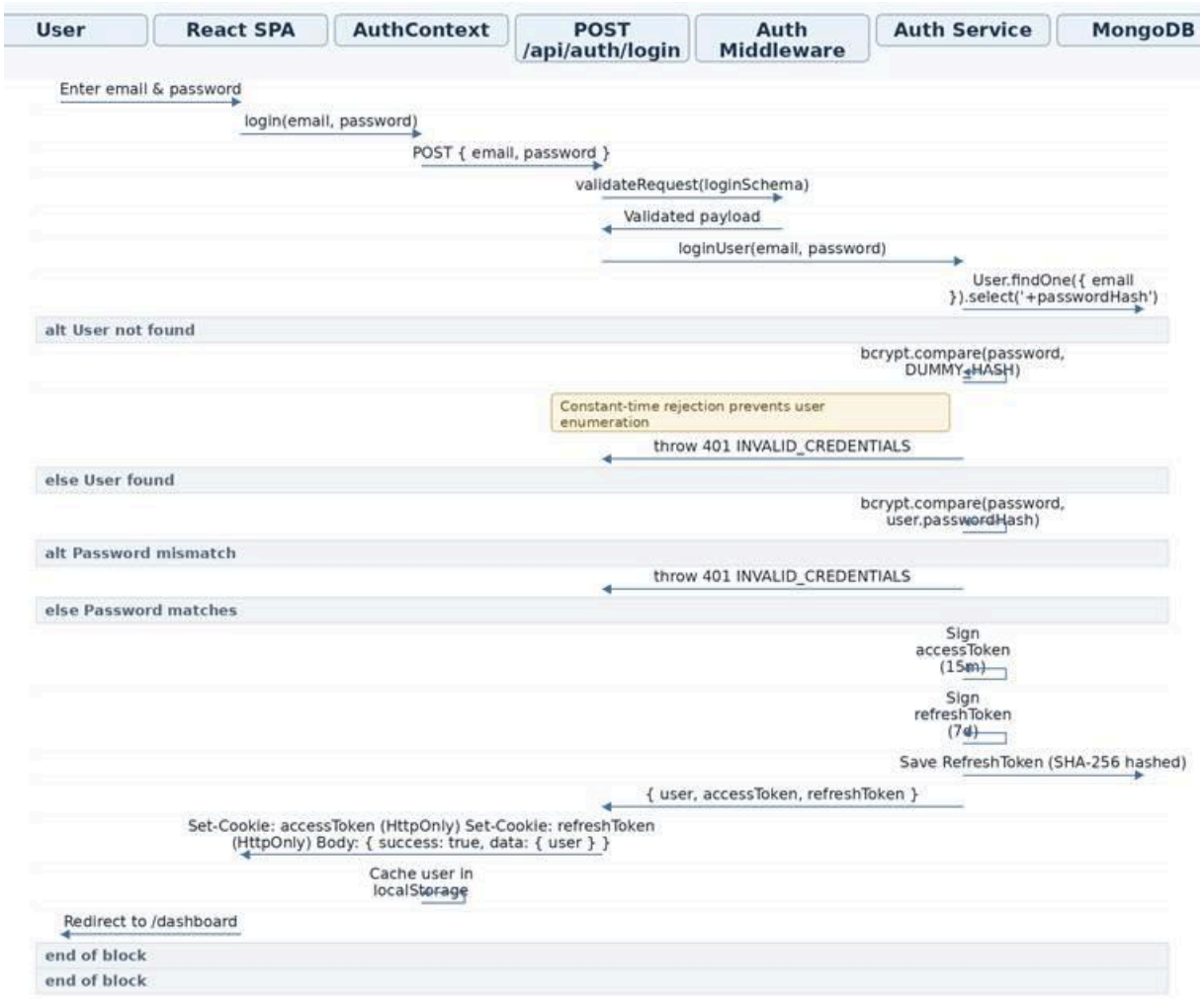
The User.preferences field is a nested sub-document with the following structure:

Field	Type	Default	Description
includeGymSubmissions	Boolean	false	Whether to include gym/virtual contest submissions in analysis
weeklyReport	Boolean	true	Automatically generate weekly performance snapshots
streakReminders	Boolean	true	Notify when practice streak is about to break
contestReminders	Boolean	true	Notify about upcoming Codeforces contests
automaticRefresh	Boolean	true	Auto-refresh analytics when data may be stale

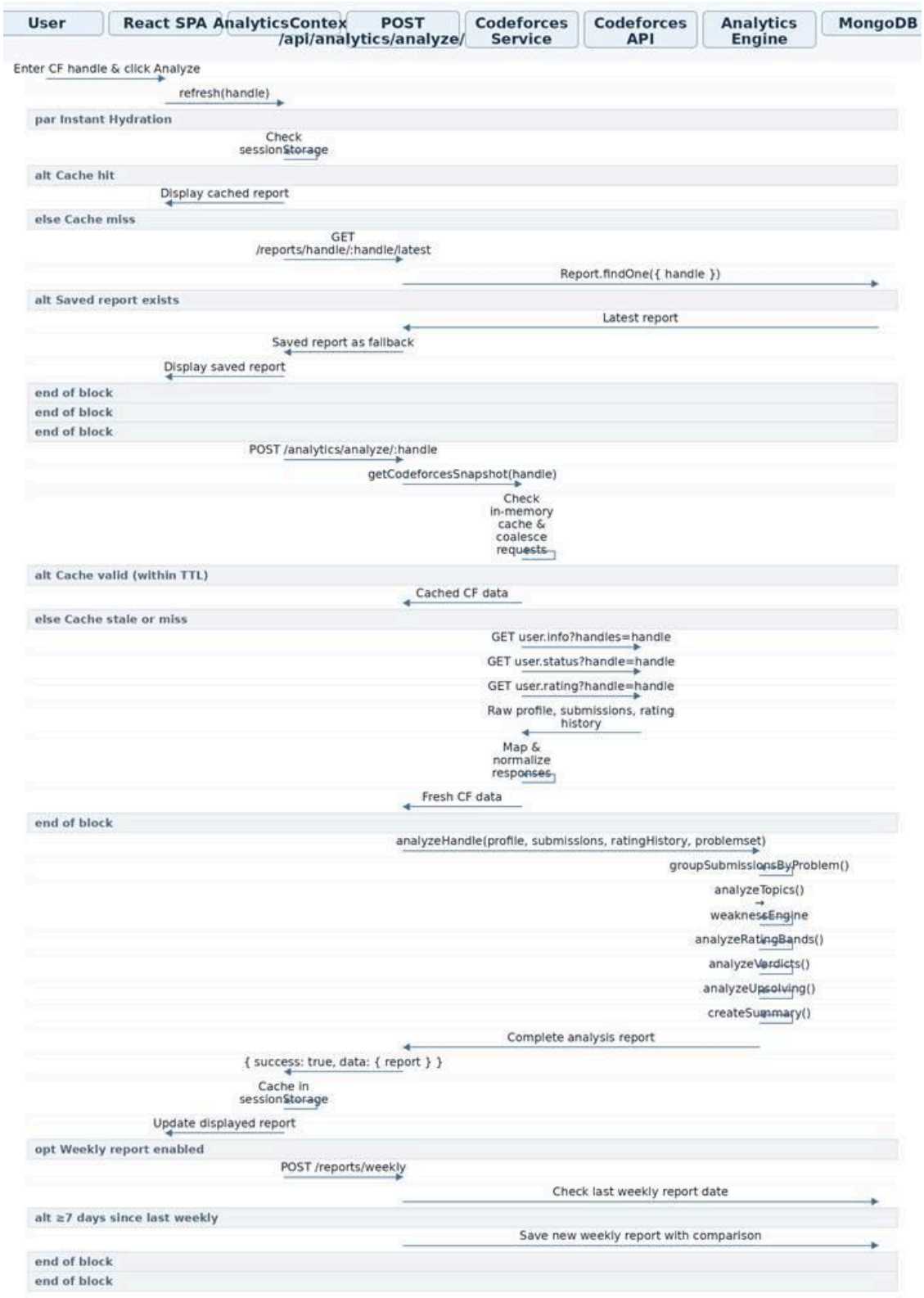
---

6. Relevant UML diagrams

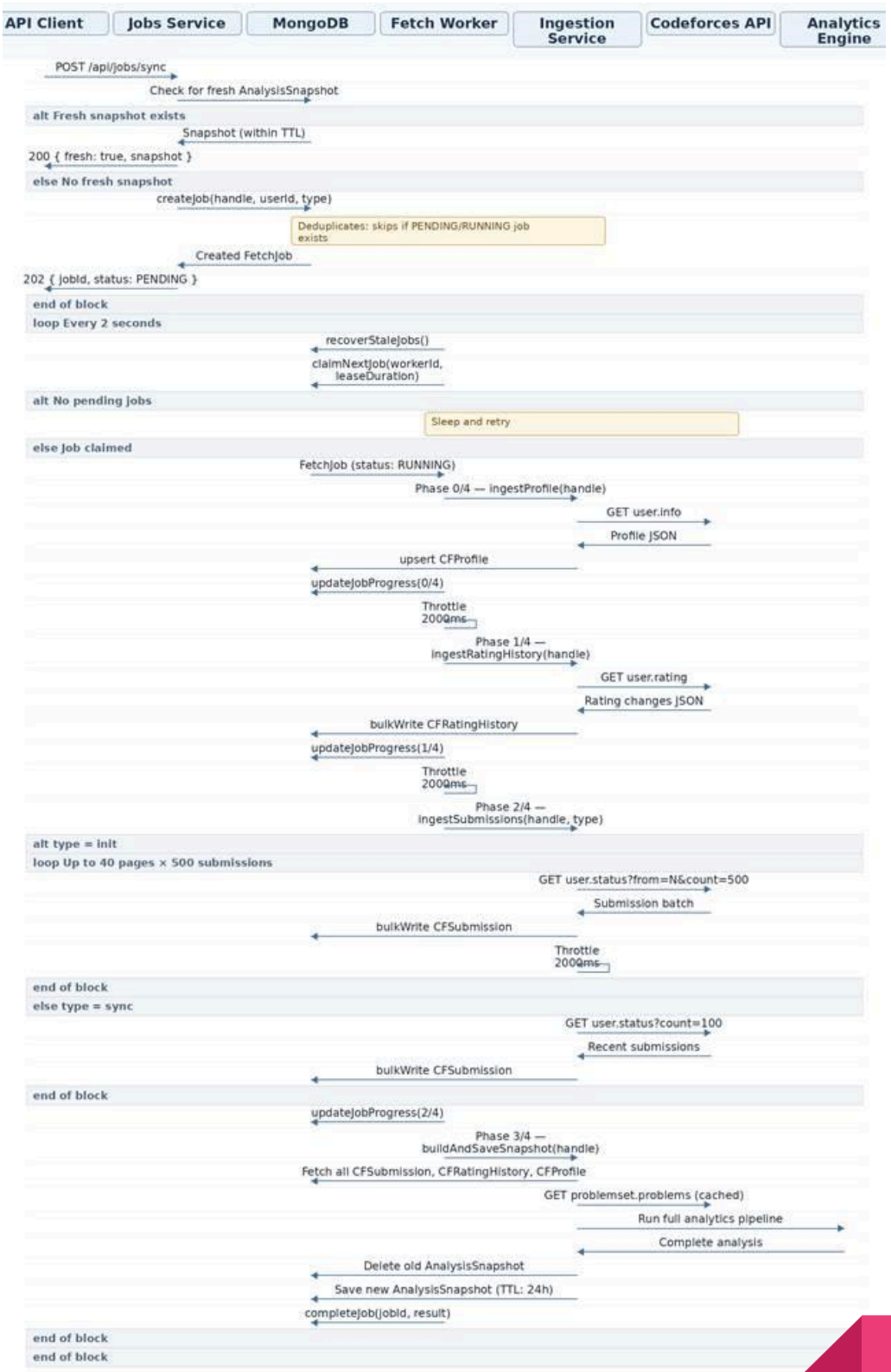
6.4.1 User Authentication (Login Flow)


User
React SPA
AuthContext
POST /api/auth/login
Auth Middleware
Auth Service
MongoDB
Enter email & password
login(email, password)
POST { email, password }
validateRequest(loginSchema)
Validated payload
loginUser(email, password)
User.findOne({ email }).select('+passwordHash')
alt User not found
bcrypt.compare(password, DUMMY_HASH)
Constant-time rejection prevents user enumeration
throw 401 INVALID_CREDENTIALS
else User found
bcrypt.compare(password, user.passwordHash)
alt Password mismatch
throw 401 INVALID_CREDENTIALS
else Password matches
Sign accessToken (15m)
Sign refreshToken (7d)
Save RefreshToken (SHA-256 hashed)
{ user, accessToken, refreshToken }
Set-Cookie: accessToken (HttpOnly) Set-Cookie: refreshToken (HttpOnly) Body: { success: true, data: { user } }
Cache user in localStorage
Redirect to /dashboard
end of block
end of block

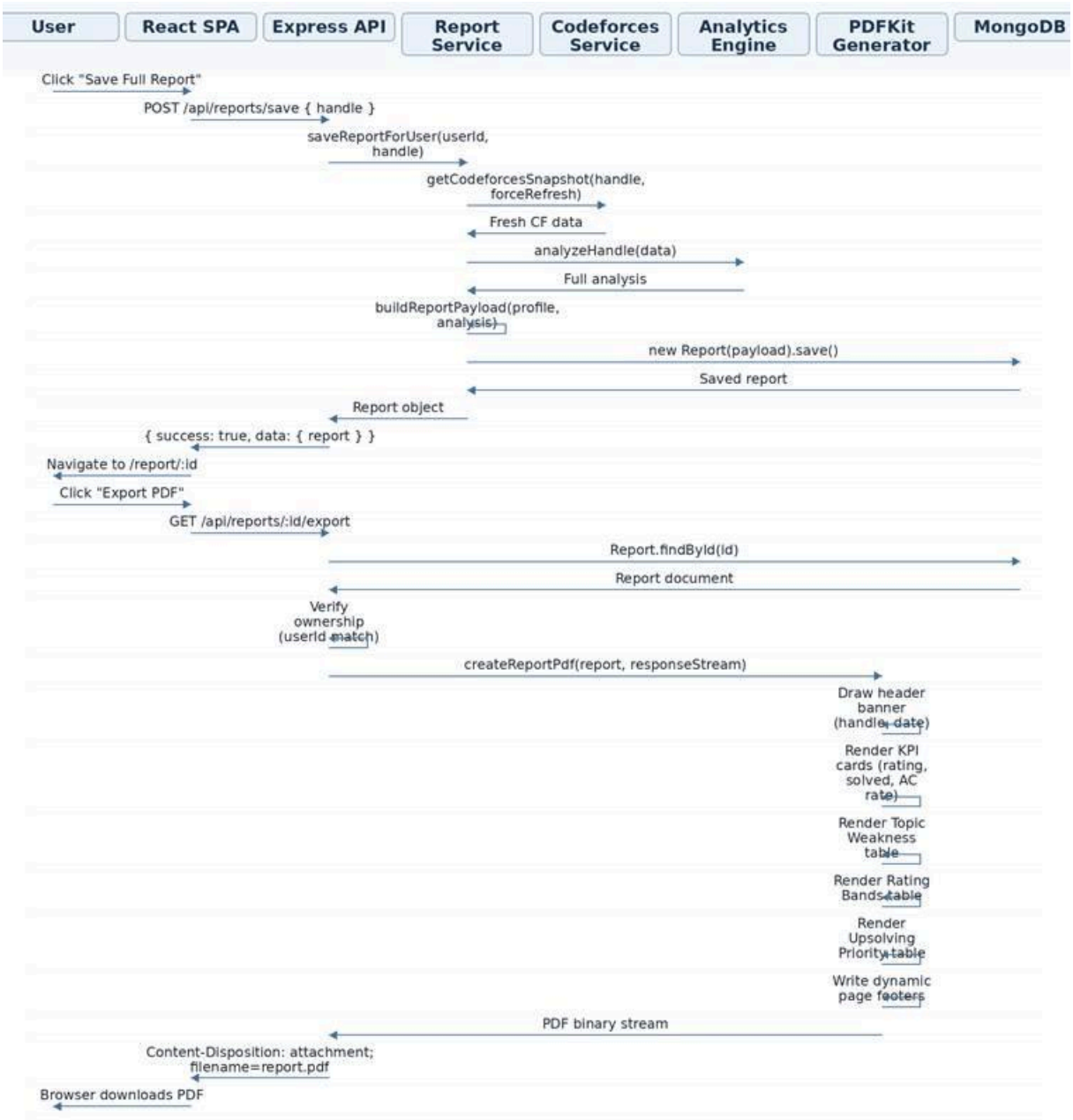
6.4.2 Codeforces Handle Analysis


User
React SPA
AnalyticsContex
POST /api/analytics/analyze/
Codeforces Service
Codeforces API
Analytics Engine
MongoDB
Enter CF handle & click Analyze
refresh(handle)
par Instant Hydration
Check sessionStorage
alt Cache hit
Display cached report
else Cache miss
GET /reports/handle/:handle/latest
Report.findOne({ handle })
alt Saved report exists
Latest report
Saved report as fallback
Display saved report
end of block
end of block
end of block
POST /analytics/analyze/:handle
getCodeforcesSnapshot(handle)
Check in-memory cache & coalesce requests
alt Cache valid (within TTL)
Cached CF data
else Cache stale or miss
GET user.info?handles=handle
GET user.status?handle=handle
GET user.rating?handle=handle
Raw profile, submissions, rating history
Map & normalize responses
Fresh CF data
end of block
analyzeHandle(profile, submissions, ratingHistory, problemset)
groupSubmissionsByProblem()
analyzeTopics()
weaknessEngine
analyzeRatingBands()
analyzeVerdicts()
analyzeUpsolving()
createSummary()
Complete analysis report
{ success: true, data: { report } }
Cache in sessionStorage
Update displayed report
opt Weekly report enabled
POST /reports/weekly
Check last weekly report date
alt ≥7 days since last weekly
Save new weekly report with comparison
end of block
end of block

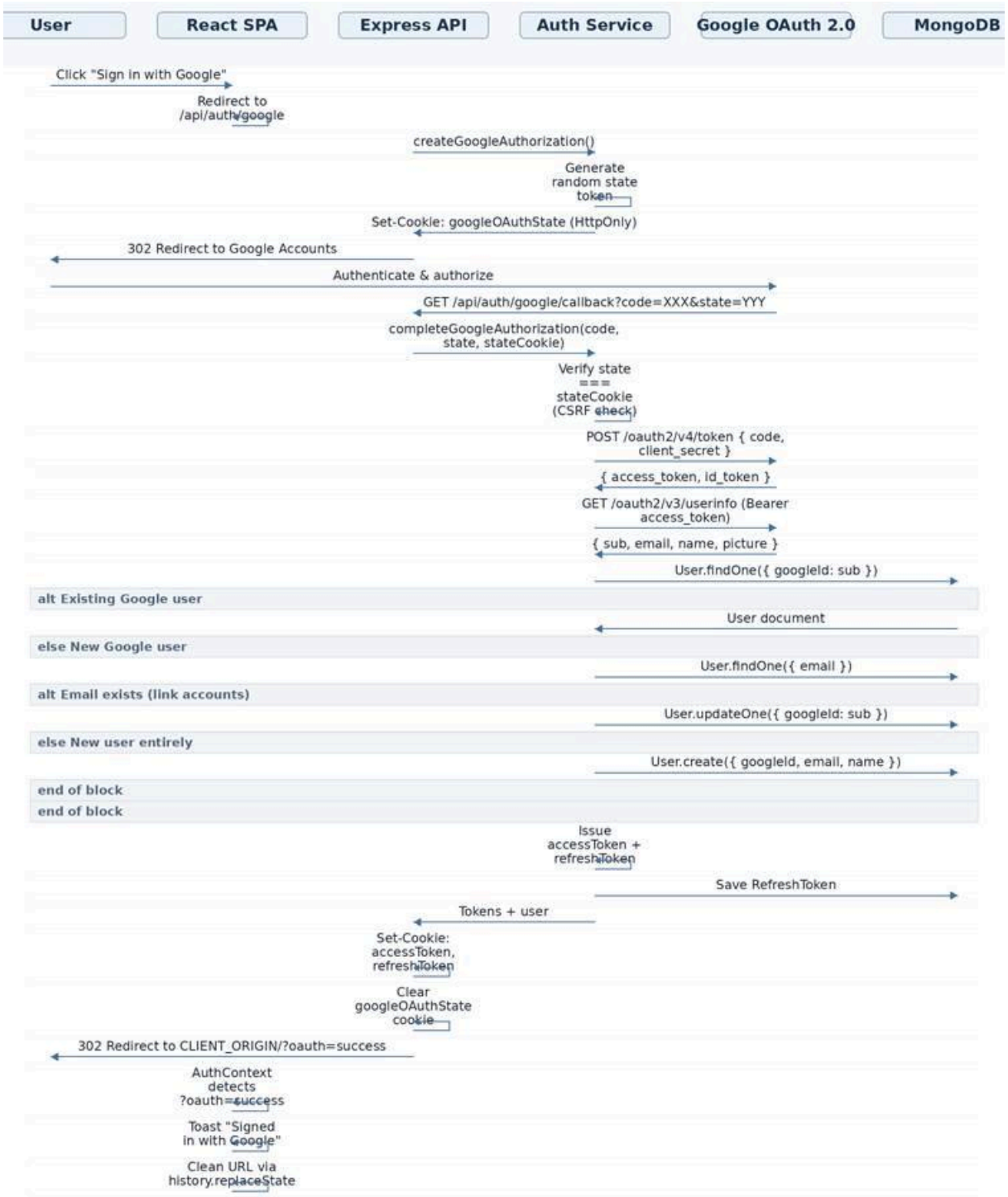
6.4.3 Background Worker Ingestion Pipeline



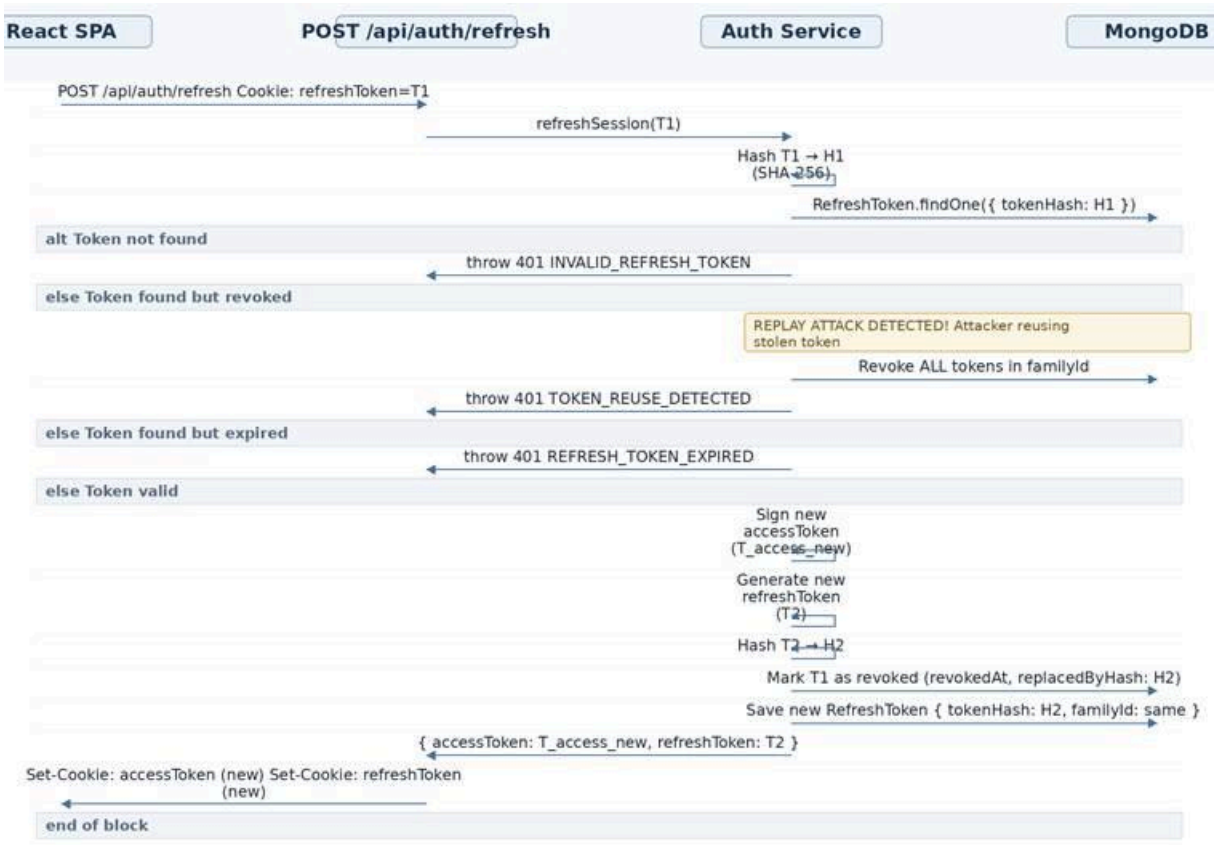
6.4.4 Report Generation & PDF Export



6.4.5 Google OAuth 2.0 Flow

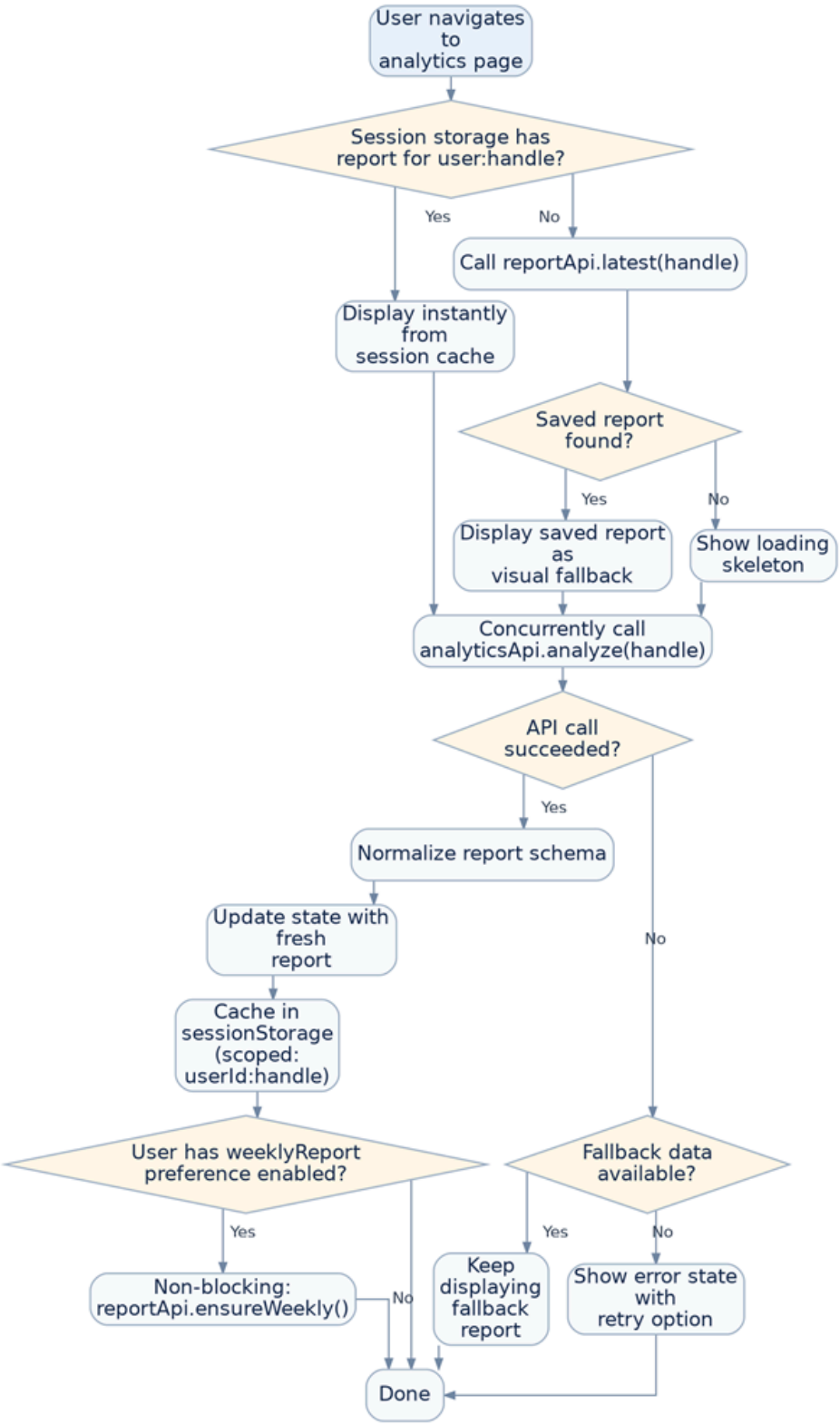

User
React SPA
Express API
Auth Service
Google OAuth 2.0
MongoDB
Click "Sign in with Google"
Redirect to /api/auth/google
createGoogleAuthorization()
Generate random state token
Set-Cookie: googleOAuthState (HttpOnly)
302 Redirect to Google Accounts
Authenticate & authorize
GET /api/auth/google/callback?code=XXX&state=YYY
completeGoogleAuthorization(code, state, stateCookie)
Verify state === stateCookie (CSRF check)
POST /oauth2/v4/token { code, client_secret }
{ access_token, id_token }
GET /oauth2/v3/userinfo (Bearer access_token)
{ sub, email, name, picture }
User.findOne({ googleId: sub })
alt Existing Google user
User document
else New Google user
User.findOne({ email })
alt Email exists (link accounts)
User.updateOne({ googleId: sub })
else New user entirely
User.create({ googleId, email, name })
end of block
end of block
Issue accessToken + refreshToken
Save RefreshToken
Tokens + user
Set-Cookie: accessToken, refreshToken
Clear googleOAuthState cookie
302 Redirect to CLIENT_ORIGIN/?oauth=success
AuthContext detects ?oauth=success
Toast "Signed in with Google"
Clean URL via history.replaceState

6.4.6 Refresh Token Rotation with Replay Detection

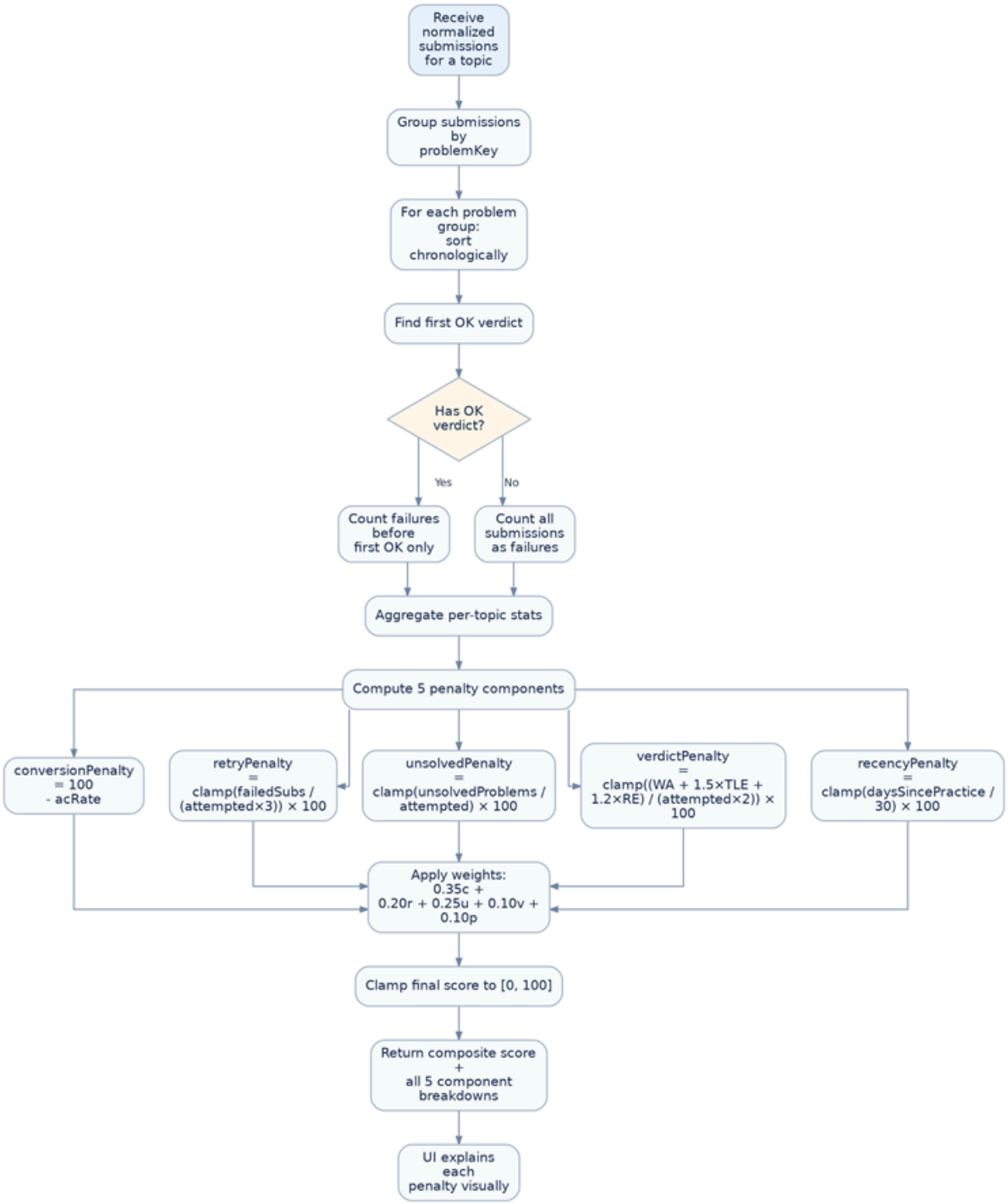


Activity Diagrams

6.4.7 Frontend 3-Tier Data Hydration

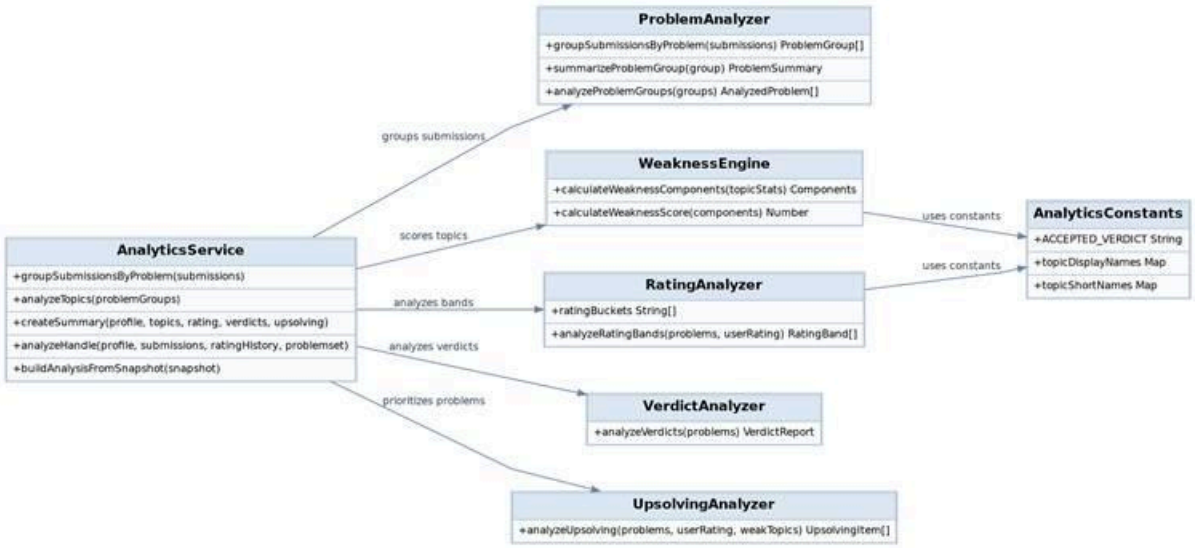

User navigates to analytics page
Session storage has report for user:handle?
Yes
No
Call reportApi.latest(handle)
Display instantly from session cache
Saved report found?
Yes
No
Display saved report as visual fallback
Show loading skeleton
Concurrently call analyticsApi.analyze(handle)
API call succeeded?
Yes
Normalize report schema
Update state with fresh report
Cache in sessionStorage (scoped: userId:handle)
No
User has weeklyReport preference enabled?
Fallback data available?
Yes
Non-blocking: reportApi.ensureWeekly()
No
Yes
No
Keep displaying fallback report
Show error state with retry option
Done

6.4.8 Weakness Score Computation Pipeline

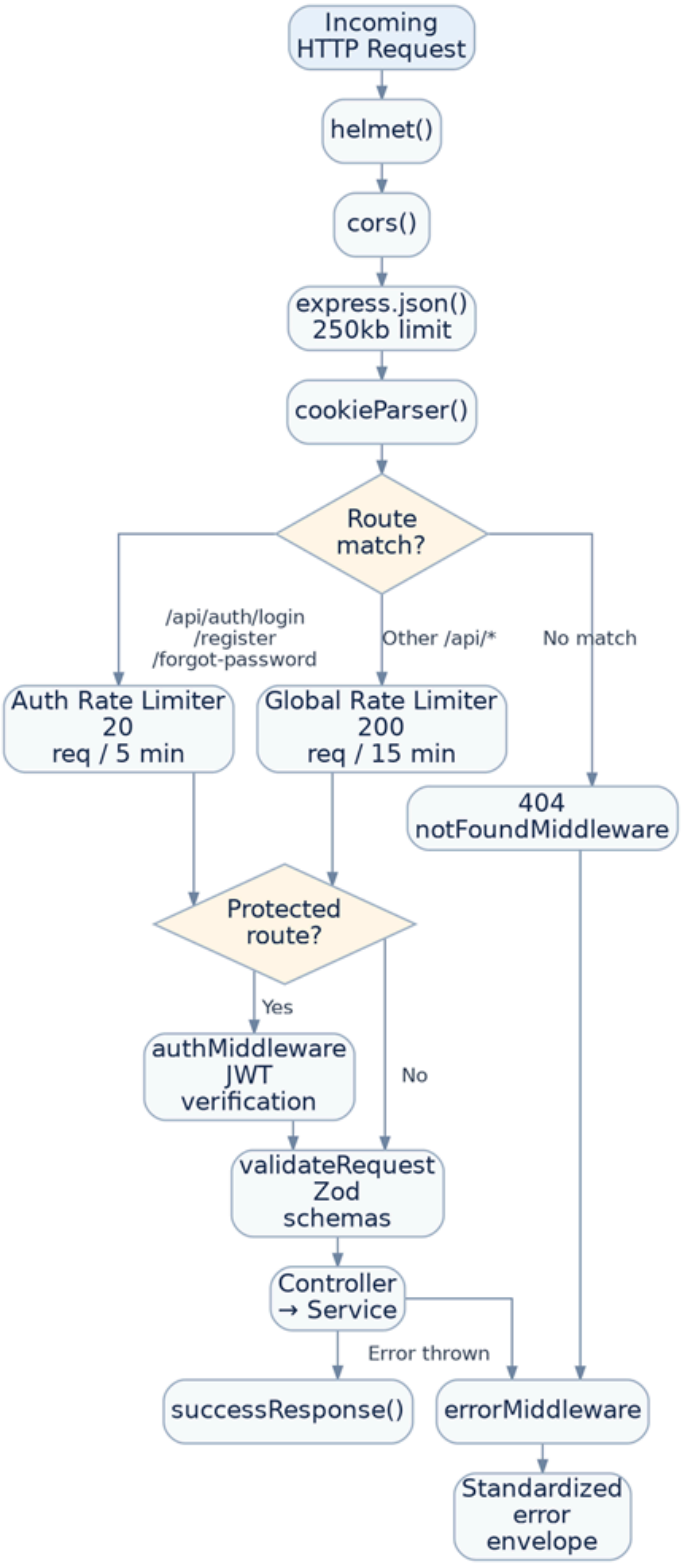


Class / Module Diagrams

6.4.9 Analytics Engine Internal Structure



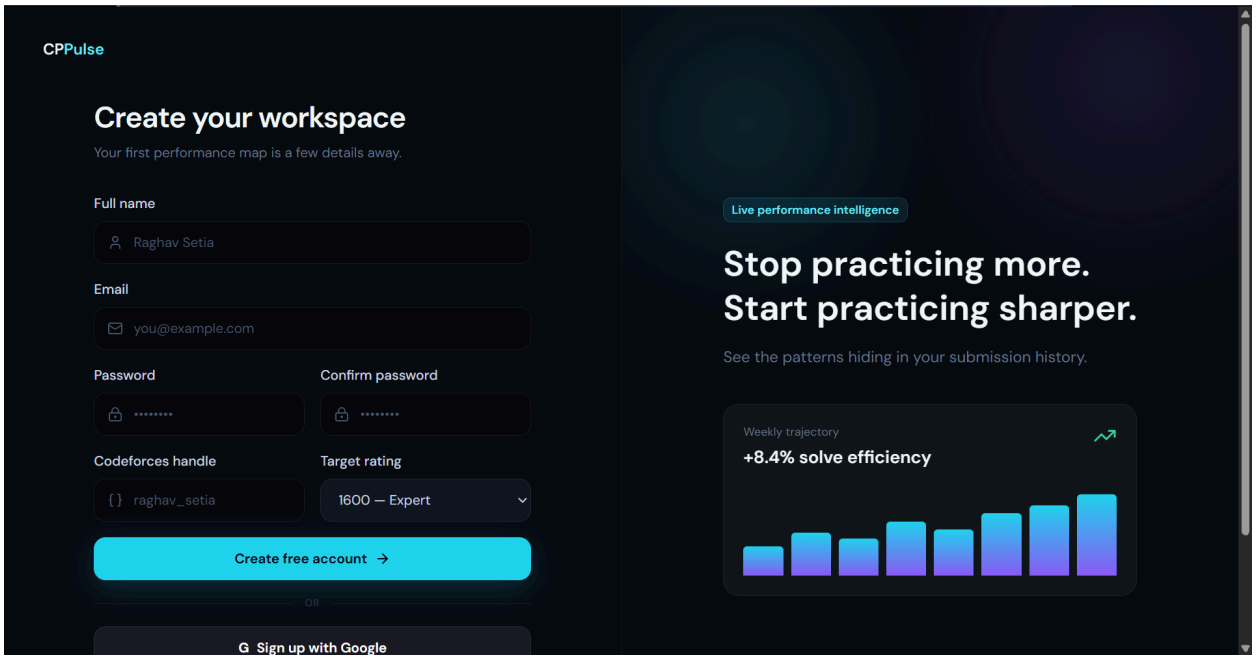
6.4.10 Express Middleware Pipeline

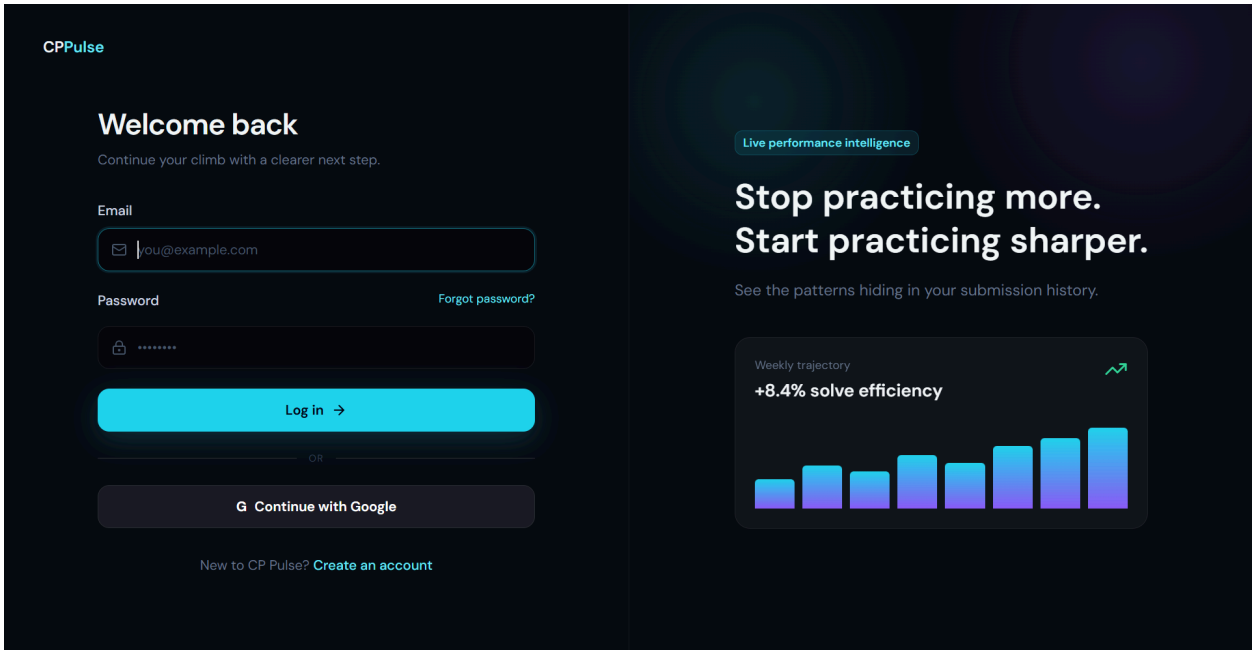

Incoming HTTP Request
helmet()
cors()
express.json() 250kb limit
cookieParser()
Route match?
/api/auth/login /register /forgot-password
Other /api/*
No match
Auth Rate Limiter 20 req / 5 min
Global Rate Limiter 200 req / 15 min
404 notFoundMiddleware
Protected route?
Yes
No
authMiddleware JWT verification
validateRequest Zod schemas
Controller → Service
Error thrown
successResponse()
errorMiddleware
Standardized error envelope

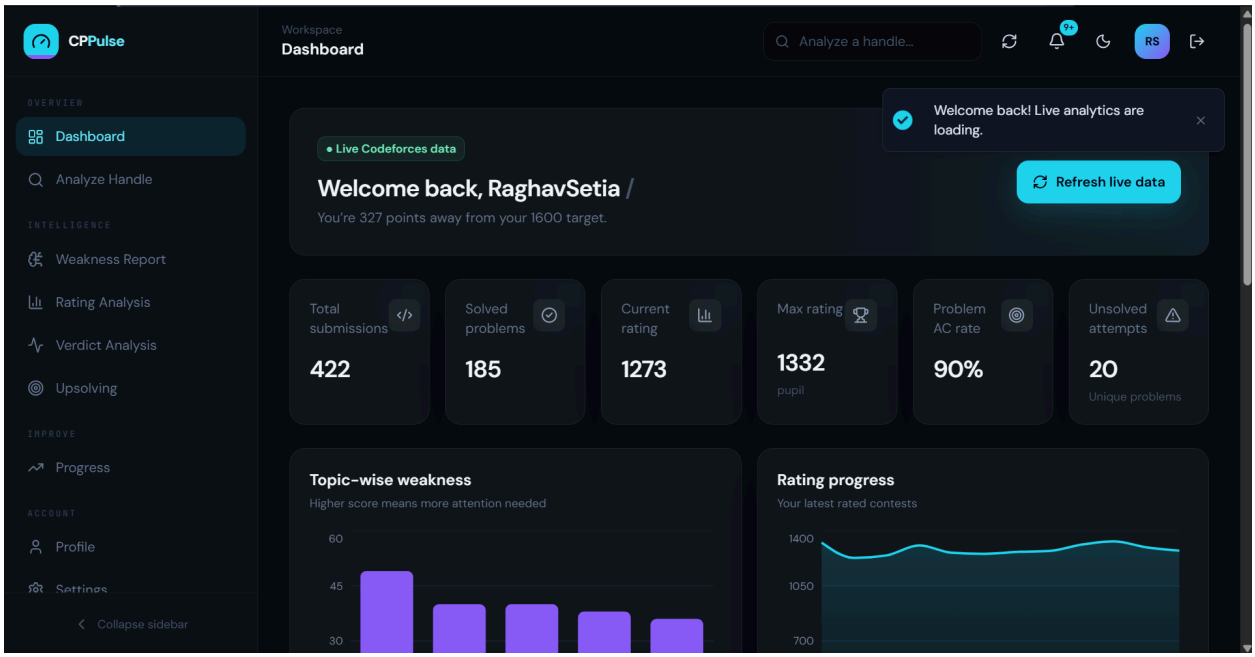
7. UI wireframes/prototype

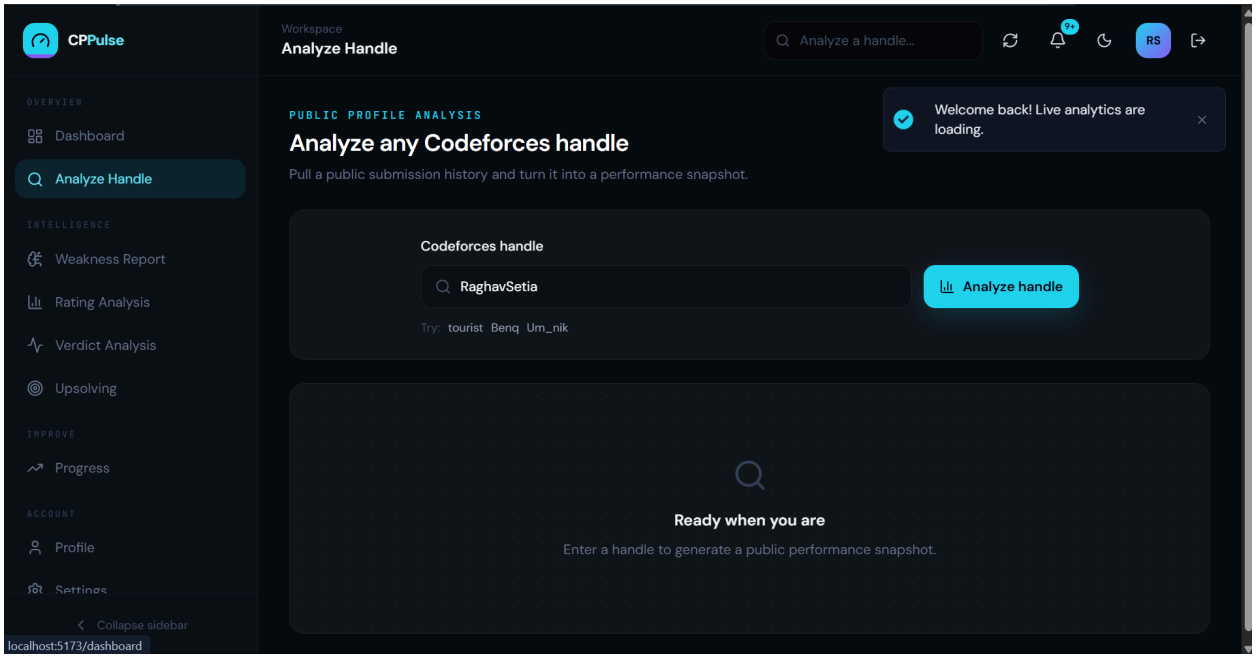
#	Screen	Description
1	Landing Page	Public marketing page with feature overview
2	Registration Page	User registration form with validation feedback
3	Login Page	Email/password login with Google OAuth option
4	Onboarding Page	Initial CF handle linking and preference setup
5	Dashboard	Main analytics dashboard with summary KPIs and charts
6	Analyze Handle	Handle input, loading state, and analysis result
7	Weakness Report	Topic weakness breakdown with component scores
8	Rating Analysis	Rating band chart with AC rates and weak tags
9	Verdict Analysis	Verdict distribution donut chart and painful problems
10	Upsolving Queue	Prioritized unsolved problems with action links
11	Progress Page	Longitudinal trends across saved reports
12	Profile / Settings	User preferences and handle management
13	Report Details	Saved report view with PDF export button
14	PDF Export	Generated PDF document sample

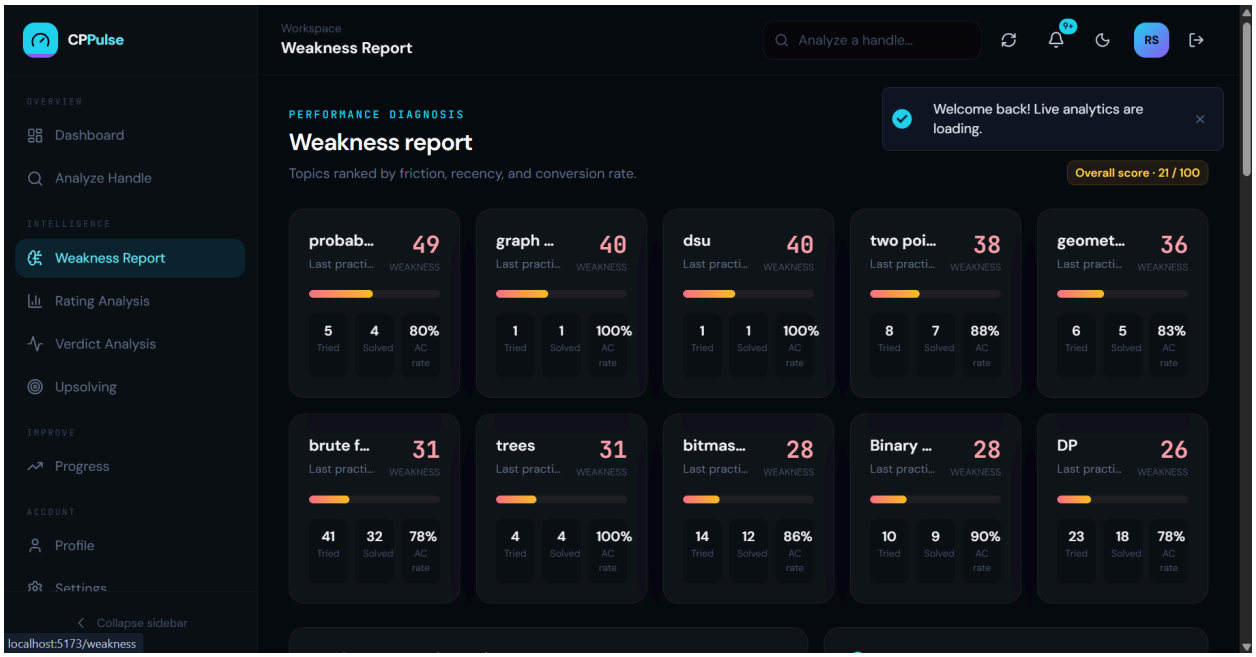

CPPulse
Features
How It Works
Demo
Pricing
Log in
Start free
Competitive programming, decoded
Turn your Codeforces history into a clear improvement roadmap.
Analyze weak topics, rating gaps, verdict patterns, and get AI-guided upsolving plans built around the way you actually compete.
Analyze my handle
View live demo
No credit card · Your first analysis takes under a minute

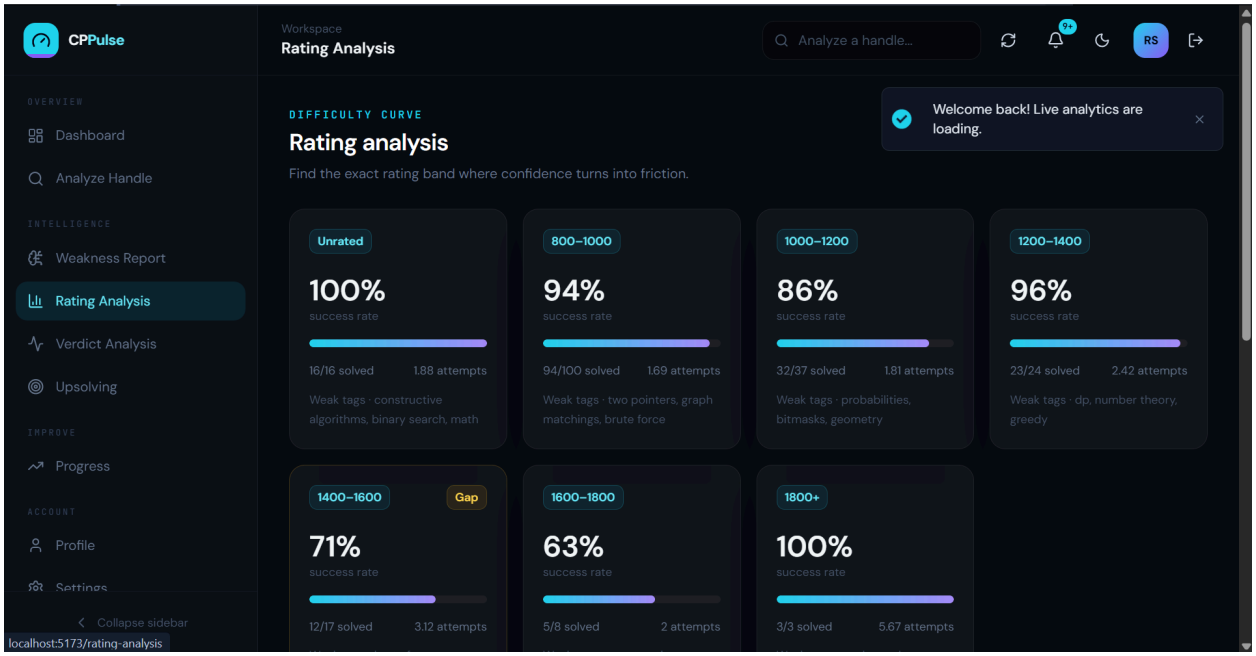

CPPulse
Create your workspace
Your first performance map is a few details away.
Full name
Raghav Setia
Email
you@example.com
Password
Confirm password
Codeforces handle
raghav_setia
Target rating
1600 — Expert
Create free account
OR
G Sign up with Google
Live performance intelligence
Stop practicing more. Start practicing sharper.
See the patterns hiding in your submission history.
Weekly trajectory
+8.4% solve efficiency

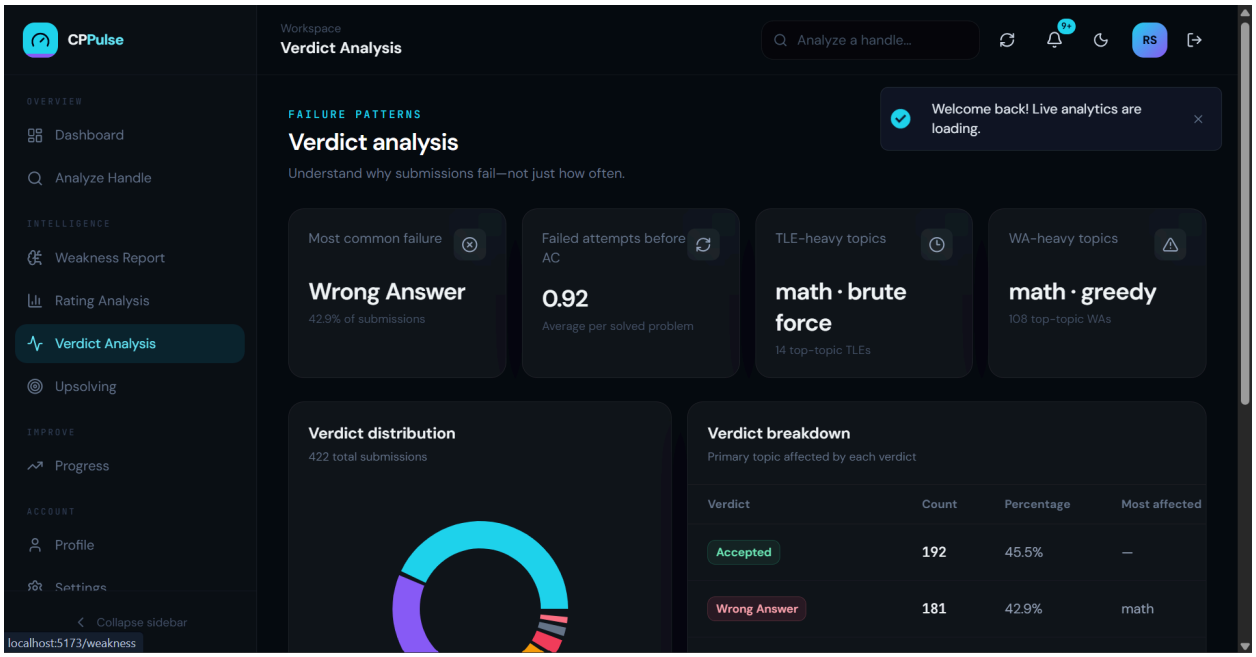

CPPulse
Welcome back
Continue your climb with a clearer next step.
Email
you@example.com
Password
Forgot password?
Log in →
OR
G Continue with Google
New to CP Pulse? Create an account
Live performance intelligence
Stop practicing more.
Start practicing sharper.
See the patterns hiding in your submission history.
Weekly trajectory
+8.4% solve efficiency

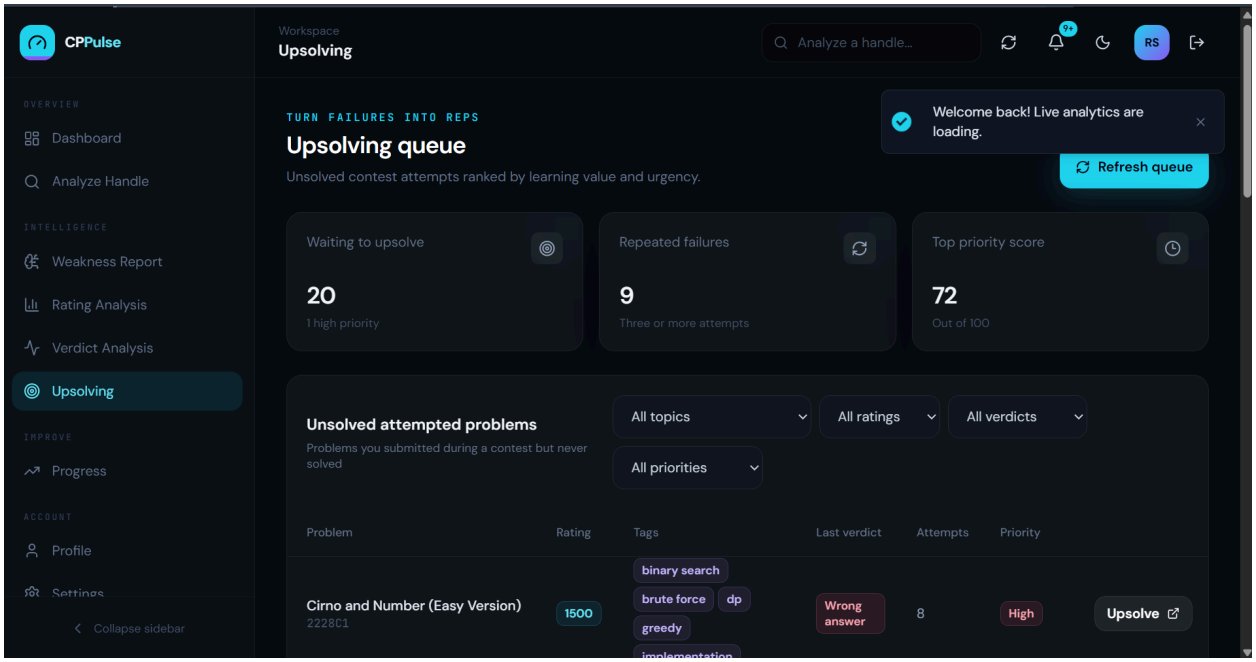

CPPulse
Workspace
Dashboard
Analyze a handle...
RS
OVERVIEW
Dashboard
Analyze Handle
INTELLIGENCE
Weakness Report
Rating Analysis
Verdict Analysis
Upsolving
IMPROVE
Progress
ACCOUNT
Profile
Settings
Collapse sidebar
Welcome back! Live analytics are loading.
Live Codeforces data
Welcome back, RaghavSetia /
You're 327 points away from your 1600 target.
Refresh live data
Total submissions
422
Solved problems
185
Current rating
1273
Max rating
1332
pupil
Problem AC rate
90%
Unsolved attempts
20
Unique problems
Topic-wise weakness
Higher score means more attention needed
60
45
30
Rating progress
Your latest rated contests
1400
1050
700

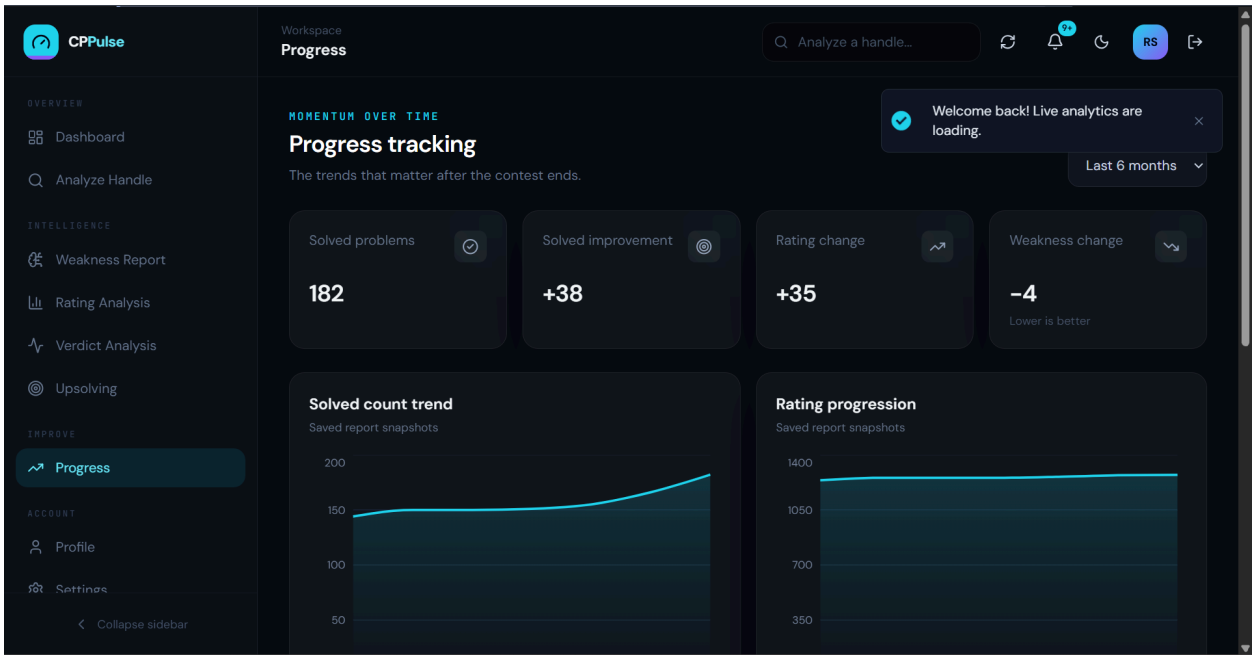

CPPulse
Workspace
Analyze Handle
Analyze a handle...
RS
OVERVIEW
Dashboard
Analyze Handle
INTELLIGENCE
Weakness Report
Rating Analysis
Verdict Analysis
Upsolving
IMPROVE
Progress
ACCOUNT
Profile
Settings
Collapse sidebar
PUBLIC PROFILE ANALYSIS
Analyze any Codeforces handle
Pull a public submission history and turn it into a performance snapshot.
Welcome back! Live analytics are loading.
Codeforces handle
RaghavSetia
Analyze handle
Try: tourist Benq Um_nik
Ready when you are
Enter a handle to generate a public performance snapshot.
localhost:5173/dashboard

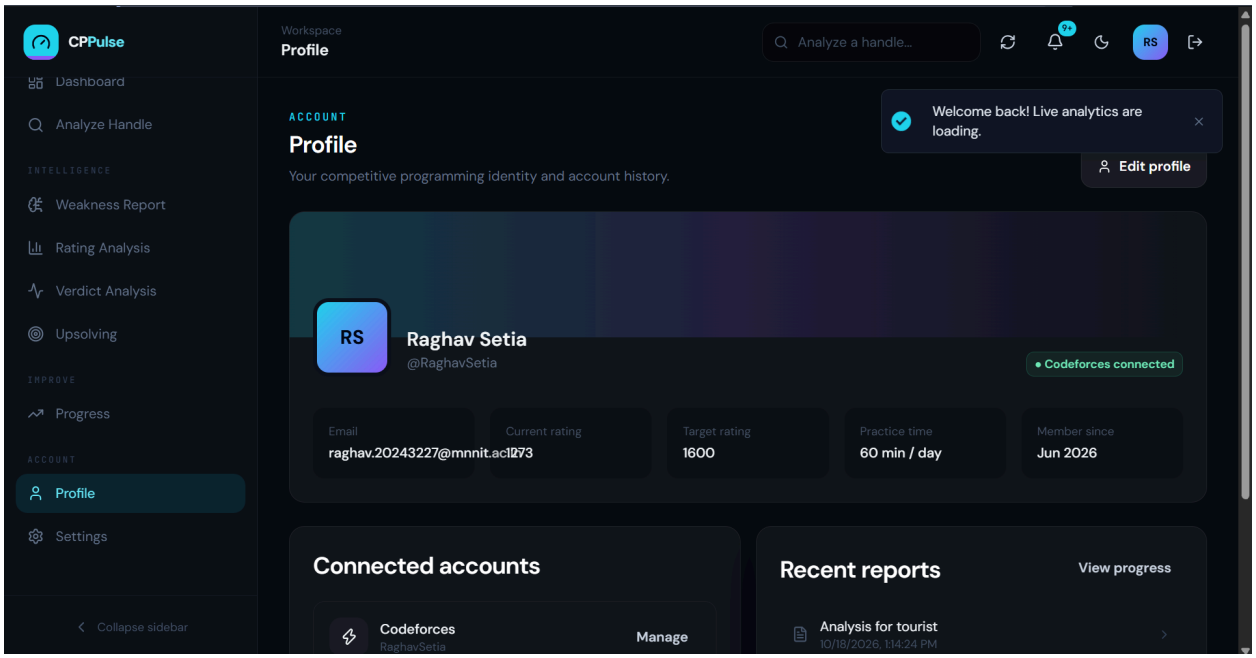

CPPulse
Workspace
Weakness Report
Analyze a handle...
RS
Dashboard
Analyze Handle
Weakness Report
Rating Analysis
Verdict Analysis
Upsolving
Progress
Profile
Settings
Collapse sidebar
PERFORMANCE DIAGNOSIS
Weakness report
Topics ranked by friction, recency, and conversion rate.
Welcome back! Live analytics are loading.
Overall score · 21 / 100
probab... 49
graph ... 40
dsu 40
two poi... 38
geomet... 36
brute f... 31
trees 31
bitmas... 28
Binary ... 28
DP 26
WEAKNESS
Tried
Solved
AC rate
localhost:5173/weakness

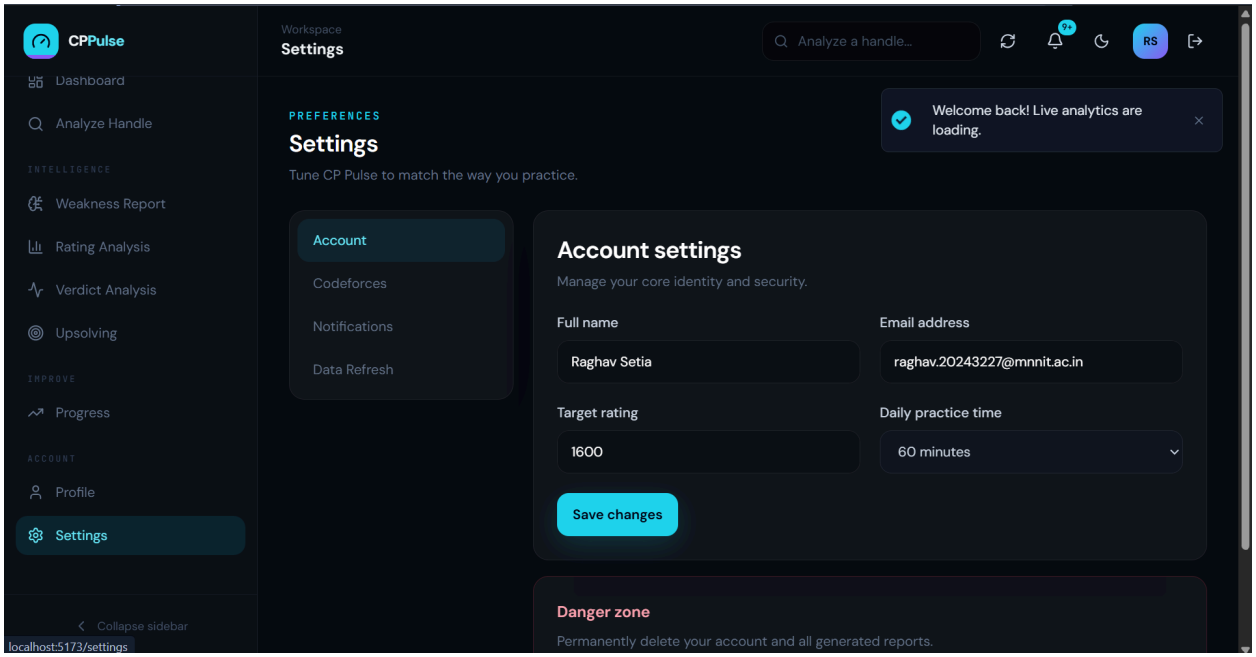

CPPulse
Workspace
Rating Analysis
Analyze a handle...
RS
OVERVIEW
Dashboard
Analyze Handle
INTELLIGENCE
Weakness Report
Rating Analysis
Verdict Analysis
Upsolving
IMPROVE
Progress
ACCOUNT
Profile
Settings
Collapse sidebar
localhost:5173/rating-analysis
DIFFICULTY CURVE
Rating analysis
Find the exact rating band where confidence turns into friction.
Welcome back! Live analytics are loading.
Unrated
100%
success rate
16/16 solved
1.88 attempts
Weak tags · constructive algorithms, binary search, math
800–1000
94%
success rate
94/100 solved
1.69 attempts
Weak tags · two pointers, graph matchings, brute force
1000–1200
86%
success rate
32/37 solved
1.81 attempts
Weak tags · probabilities, bitmasks, geometry
1200–1400
96%
success rate
23/24 solved
2.42 attempts
Weak tags · dp, number theory, greedy
1400–1600
Gap
71%
success rate
12/17 solved
3.12 attempts
1600–1800
63%
success rate
5/8 solved
2 attempts
1800+
100%
success rate
3/3 solved
5.67 attempts

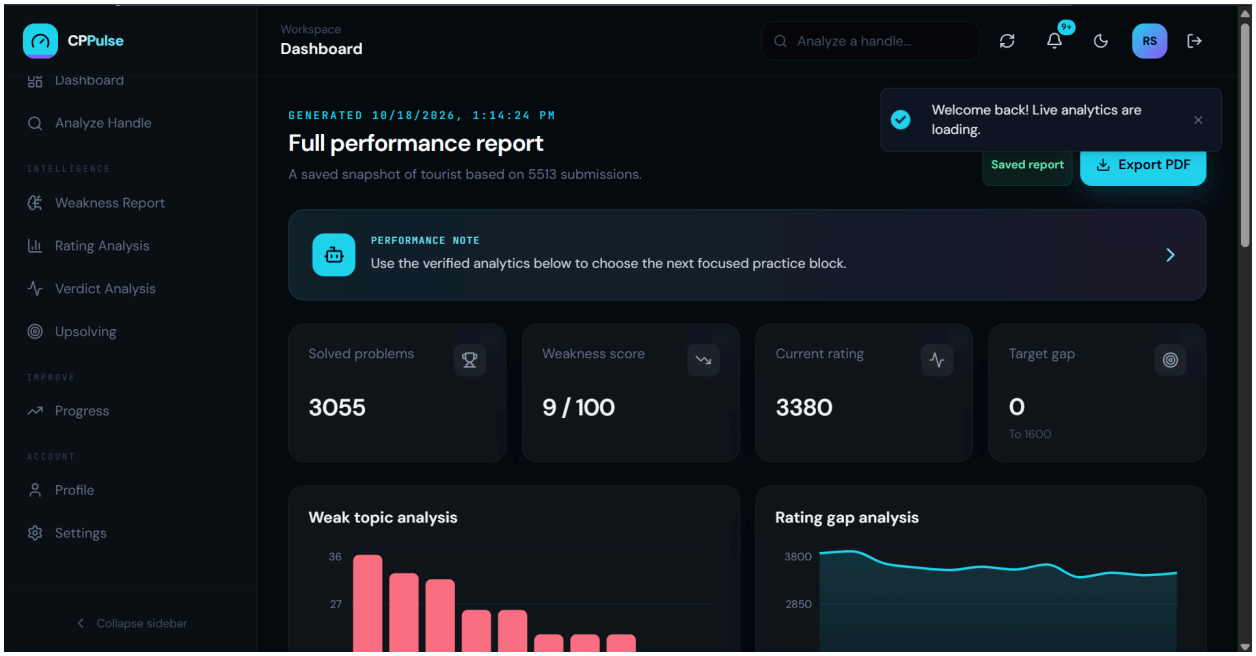

CPPulse
Workspace
Verdict Analysis
Analyze a handle...
RS
OVERVIEW
Dashboard
Analyze Handle
INTELLIGENCE
Weakness Report
Rating Analysis
Verdict Analysis
Upsolving
IMPROVE
Progress
ACCOUNT
Profile
Settings
Collapse sidebar
localhost:5173/weakness
FAILURE PATTERNS
Verdict analysis
Understand why submissions fail—not just how often.
Welcome back! Live analytics are loading.
Most common failure
Wrong Answer
42.9% of submissions
Failed attempts before AC
0.92
Average per solved problem
TLE-heavy topics
math · brute force
14 top-topic TLEs
WA-heavy topics
math · greedy
108 top-topic WAs
Verdict distribution
422 total submissions
Verdict breakdown
Primary topic affected by each verdict
Verdict
Count
Percentage
Most affected
Accepted
192
45.5%
—
Wrong Answer
181
42.9%
math


CPPulse
Workspace
Upsolving
Analyze a handle...
RS
OVERVIEW
Dashboard
Analyze Handle
INTELLIGENCE
Weakness Report
Rating Analysis
Verdict Analysis
Upsolving
IMPROVE
Progress
ACCOUNT
Profile
Settings
Collapse sidebar
TURN FAILURES INTO REPS
Upsolving queue
Unsolved contest attempts ranked by learning value and urgency.
Welcome back! Live analytics are loading.
Refresh queue
Waiting to upsolve
20
1 high priority
Repeated failures
9
Three or more attempts
Top priority score
72
Out of 100
Unsolved attempted problems
Problems you submitted during a contest but never solved
All topics
All ratings
All verdicts
All priorities
Problem
Rating
Tags
Last verdict
Attempts
Priority
Cirno and Number (Easy Version)
2228C1
1500
binary search
brute force
dp
greedy
Wrong answer
8
High
Upsolve


CPPulse
Workspace
Progress
Analyze a handle...
RS
OVERVIEW
Dashboard
Analyze Handle
INTELLIGENCE
Weakness Report
Rating Analysis
Verdict Analysis
Upsolving
IMPROVE
Progress
ACCOUNT
Profile
Settings
Collapse sidebar
MOMENTUM OVER TIME
Progress tracking
The trends that matter after the contest ends.
Welcome back! Live analytics are loading.
Last 6 months
Solved problems
182
Solved improvement
+38
Rating change
+35
Weakness change
-4
Lower is better
Solved count trend
Saved report snapshots
200
150
100
50
Rating progression
Saved report snapshots
1400
1050
700
350


CPPulse
Workspace
Profile
Analyze a handle...
RS
Dashboard
Analyze Handle
INTELLIGENCE
Weakness Report
Rating Analysis
Verdict Analysis
Upsolving
IMPROVE
Progress
ACCOUNT
Profile
Settings
Collapse sidebar
ACCOUNT
Profile
Your competitive programming identity and account history.
Welcome back! Live analytics are loading.
Edit profile
RS
Raghav Setia
@RaghavSetia
Codeforces connected
Email
raghav.20243227@mnnit.ac1273
Current rating
Target rating
1600
Practice time
60 min / day
Member since
Jun 2026
Connected accounts
Codeforces
RaghavSetia
Manage
Recent reports
View progress
Analysis for tourist
10/18/2026, 1:14:24 PM


CPPulse
Workspace
Settings
Analyze a handle...
RS
Dashboard
Analyze Handle
INTELLIGENCE
Weakness Report
Rating Analysis
Verdict Analysis
Upsolving
IMPROVE
Progress
ACCOUNT
Profile
Settings
Collapse sidebar
localhost:5173/settings
PREFERENCES
Settings
Tune CP Pulse to match the way you practice.
Welcome back! Live analytics are loading.
Account
Codeforces
Notifications
Data Refresh
Account settings
Manage your core identity and security.
Full name
Raghav Setia
Email address
raghav.20243227@mnnit.ac.in
Target rating
1600
Daily practice time
60 minutes
Save changes
Danger zone
Permanently delete your account and all generated reports.



8. Testing Performed

The project currently has 42 automated tests implemented using Vitest (unit) and Supertest (integration). Comprehensive manual testing and end-to-end testing are planned for the later stages of the project.

Note: Full testing with detailed evidence, screenshots, and edge-case coverage will be performed and documented in the next phase. The test cases below represent the automated test suite currently implemented in the codebase.

8.1 Test Case Table

ID	Type	Test Case	Input / Scenario	Expected Result	Actual Result	Status
TC-01	Integration	Health Check Endpoint	GET /api/health	HTTP 200 with {success: true, data: {status: "ok"}}	HTTP 200 with {success: true, data: {status: "ok"}}	Pass

TC-02	Integration	Not-Found Error Envelope	GET /api/does-not-exist	HTTP 404 with error.code = "ROUTE_NOT_FOUND"	HTTP 404 with error.code = "ROUTE_NOT_FOUND"	Pass
TC-03	Integration	Registration Validation Rejection	POST /api/auth/register with {name: "R", email: "not-an-email", password: "short"}	HTTP 400 with error.code = "VALIDATION_ERROR"	HTTP 400 with error.code = "VALIDATION_ERROR"	Pass
TC-04	Integration	Google OAuth Status Check	GET /api/auth/google/status	HTTP 200 with data.configured as boolean	HTTP 200 with data.configured as boolean	Pass
TC-05	Integration	Forgot-Password Validation	POST /api/auth/forgot-password with {email: "not-an-email"}	HTTP 400 with error.code = "VALIDATION_ERROR"	HTTP 400 with error.code = "VALIDATION_ERROR"	Pass
TC-06	Integration	Reset-Password Validation	POST /api/auth/reset-password with {token: "short", password: "short"}	HTTP 400 with error.code = "VALIDATION_ERROR"	HTTP 400 with error.code = "VALIDATION_ERROR"	Pass
TC-07	Unit	Problem Grouping by Identity	4 submissions for 2 unique problems	2 groups; first group has 3 submissions	2 groups; first group has 3 submissions	Pass
TC-08	Unit	Topic Statistics & Post-AC Exclusion	Submissions fixture with DP tag	DP: attempted=2, solved=1, unsolved=1, failed=2, rate=50%, avgAttemptsBeforeAc =2, weakness 0-100	DP: attempted=2, solved=1, unsolved=1, failed=2, rate=50%, avgAttemptsBeforeA	Pass

					c=2, weakness in [0,100]	
TC-09	Unit	Summary Calculation	Fixture submissions + empty rating history	totalSubmissions=4, attempted=2, solved=1, unsolved=1, acRate=50, recentActivity length=4	totalSubmissions=4, attempted=2, solved=1, unsolved=1, acRate=50, recentActivity length=4	Pass
TC-10	Unit	Snapshot Coalescing & TTL Reuse	3 concurrent calls to getCodeforcesSnapshot("cache_fixture")	Only 1 upstream API call; third call returns cached=true	Only 1 upstream API call; third call returns cached=true	Pass
TC-11	Unit	Problemset Normalization & Cache	2 sequential calls to getCodeforcesProblemset()	problemKey="42-A", tags=["math"] (lowercase), second call uses cache	problemKey="42-A", tags=["math"] (lowercase), second call uses cache	Pass
TC-12	Unit	Profile Mapping	CF profile {handle: "tourist", rating: 3900, maxRating: 4000}	Internal contract with same handle, rating, maxRating	Mapped with handle="tourist", rating=3900, maxRating=4000	Pass
TC-13	Unit	Submission Mapping & Tag Normalization	CF submission with tags ["Math", "math"] and contestId 123	problemKey="123-A", tags=["math"] (deduped, lowercase), URL contains /123/A	problemKey="123-A", tags=["math"], URL contains /123/A	Pass
TC-14	Unit	Rating Bucket Coverage	Fixture submissions across rating bands	All 7 buckets present: unrated, 800-1000, 1000-1200, 1200-1400, 1400-1600, 1600-1800, 1800+	All 7 buckets returned in correct order	Pass
TC-15	Unit	Bucket Aggregation	Submissions in 1000-1200 band (1	1000-1200: attempted=1,	1000-1200: attempted=1,	Pass

		Accuracy	problem, 1 solved)	solved=1, failed=1, rate=100%, avg=2	solved=1, failed=1, rate=100%, avg=2	
TC-16	Unit	Report Payload Schema	analysisFixture() → buildReportPayload("user-1")	Payload has userId, handle="fixture", isSaved=true, metadata.analyticsVersion=1	Payload has userId, handle="fixture", isSaved=true, metadata.analyticsVersion=1	Pass
TC-17	Unit	Report Persistence (Injected Model)	saveReportForUser with mock model.create	Returns saved report with id="report-1"	Returns {id: "report-1"}	Pass
TC-18	Unit	PDF Binary Validation	createReportPdf → PassThrough stream	Output starts with "%PDF" and > 1000 bytes	Output starts with "%PDF", length > 1000 bytes	Pass
TC-19	Unit	Weekly Report Generation	Previous report from 2026-06-01, current date 2026-06-10	generated=true, reportType="weekly", weeklyComparison with hasBaseline, deltas	generated=true, reportType="weekly", solvedChange=2, ratingChange=20, weaknessChange=-10	Pass
TC-20	Unit	Force Fresh Snapshot on Save	saveReportForUser → check analyze() call args	analyze called with {forceRefresh: true}	analyze called with {forceRefresh: true}	Pass
TC-21	Unit	Case-Insensitive Handle Lookup	getLatestReportForHandle("user-1", "Raghav")	Filter uses RegExp matching "raghav" and "RAGHAV"	RegExp matches both "raghav" and "RAGHAV"	Pass
TC-22	Unit	Weekly Report 7-Day Deduplication	Existing weekly from 2026-06-08, current date 2026-06-10	generated=false, returns existing report, analyze not called	generated=false, returns existing report, analyze not called	Pass

TC-23	Unit	Progress Case-Insensitive Match	getProgressForUser("user-1", "Raghav")	Filter uses RegExp matching both cases	RegExp matches both "raghav" and "RAGHAV"	Pass
TC-24	Unit	Progress Trend Confidence (< 2 Reports)	buildProgressHistory with 1 report	hasEnoughData = false	hasEnoughData = false	Pass
TC-25	Unit	Progress Trend Computation (2 Reports)	buildProgressHistory with 2 reports (DP weakness 70 → 55)	hasEnoughData = true, topicImprovement[0].improvement = 15	hasEnoughData = true, topicImprovement[0].improvement = 15	Pass
TC-26	Unit	Verdict Distribution	Fixture submissions	totalSubmissions=4, mostCommonFailedVerdict=WRONG_ANSWER, avgFailedAttempts=1	totalSubmissions=4, mostCommonFailedVerdict.key=WRONG_ANSWER, avgFailedAttemptsBeforeAc=1	Pass
TC-27	Unit	Upsolving Filter & Priority	Submissions + profile {rating: 1100} + weakness [{DP: 80}]	Only unsolved problems returned, problemKey="200-B", priorityScore > 0	1 result: problemKey="200-B", attempts=1, lastVerdict=WRONG_ANSWER, priorityScore > 0	Pass

8.2 Evidence / Screenshots

[To be added in the next phase. Will include terminal output of test runs, coverage reports, and screenshots of manual testing scenarios.]